

LAPORAN PRAKTIKUM
PRAKTIKUM PENAMBANGAN DATA
PERTEMUAN 3
CLASSIFICATION



Disusun oleh:

Nama : Hafidz Rizqullah Prasetya
NIM : 24/535493/SV/24243
Kelas : PL5A1
Dosen Pengampu : Dr. Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 Classification	4
2.2 Algoritma Classification	4
2.2.1 Logistic Regression	4
2.2.2 K-Nearest Neighbors (KNN)	5
2.2.3 Decision Tree	6
2.2.4 Random Forest	8
2.2.5 AdaBoost (Adaptive Boosting)	10
2.3 Metrik Evaluasi Classification	10
2.3.1 Confusion Matrix	10
2.3.2 Accuracy, Precision, Recall, F1-Score	11
2.4 Validation Techniques	12
Hasil dan Pembahasan	14
3.1 Percobaan Latihan Praktikum	14
3.1.1 Persiapan dan Impor Pustaka	14
3.1.2 Memuat Dataset Stroke	15
3.1.3 Exploratory Data Analysis (EDA)	16
3.1.3.1 Deskripsi Statistik	16
3.1.3.2 Presentase Target Kelas	17
3.1.3.3 Histogram Variabel Numerik	18
3.1.3.4 Pengecekan Missing Value	20
3.1.3.5 Pengecekan Atribut Kategorikal	21
3.1.4 Data Preprocessing	21
3.1.5 Modelling	27
3.1.5.1 a. Logistic Regression	27
3.1.5.2 b. K-Nearest Neighbors (KNN)	34
3.1.5.3 c. Decision Tree	37
3.1.5.4 d. Random Forest	40

3.1.5.5	e. AdaBoost	43
3.1.6	Tugas dan Analisis: Loan Status Prediction	46
3.1.6.1	Tujuan Penggunaan Dataset	46
3.1.6.2	Atribut Input dan Output	47
3.1.6.3	Eksplorasi Awal dan Preprocessing Dataset Loan	47
3.1.6.4	Pemodelan Klasifikasi pada Dataset Loan	50
3.1.6.5	Tabel Performa Model	54
3.1.6.6	Analisis Model Terbaik	55
Kesimpulan		56
Daftar Pustaka		57

Tujuan Praktikum

Sesuai dengan Modul Pertemuan 3, tujuan pembelajaran pada praktikum ini adalah:

1. Mahasiswa mampu memahami konsep dasar beberapa algoritma klasifikasi, seperti Logistic Regression, KNN, Decision Tree, dan Random Forest.
2. Mahasiswa mengetahui fungsi dan peran metrik evaluasi klasifikasi, seperti *accuracy*, *precision*, dan *recall* dalam menilai performa model.
3. Mahasiswa dapat menerapkan berbagai algoritma klasifikasi untuk membangun model.
4. Mahasiswa mampu melakukan evaluasi model dengan menyusun tabel performa berisi *accuracy*, *precision*, dan *recall* dari setiap model.

Dasar Teori

2.1 Classification

Klasifikasi merupakan salah satu teknik *supervised learning* yang bertujuan untuk memprediksi label atau kategori dari sebuah sampel berdasarkan fitur yang dimiliki. Model klasifikasi belajar dari data berlabel, kemudian menghasilkan suatu fungsi yang mampu menentukan kelas dari data baru yang belum pernah dilihat. Klasifikasi bekerja dengan cara mencari pola dari data historis untuk menentukan keputusan kelas yang paling mendekati. Contoh penerapannya meliputi deteksi spam, diagnosis medis, hingga rekomendasi konten [1, 13].

Pada klasifikasi, dataset terdiri dari fitur input X dan label target y yang bersifat kategorikal (misalnya $y \in \{0, 1\}$ untuk klasifikasi biner). Model dilatih pada data historis untuk meminimalkan kesalahan klasifikasi, kemudian diuji pada data yang belum pernah dilihat untuk mengukur kemampuan generalisasinya. Karena target bersifat diskrit, evaluasi tidak cukup menggunakan error regresi, melainkan menggunakan matriks kebingungan dan metrik turunan seperti *accuracy*, *precision*, *recall*, dan *F1-score* [5, 12].

2.2 Algoritma Classification

Ada banyak metode yang dapat dipakai untuk melakukan klasifikasi. Modul Pertemuan 3 menekankan lima algoritma yang paling sering digunakan karena sederhana, efektif, dan terdokumentasi baik di Scikit-Learn: Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, dan AdaBoost.

2.2.1 Logistic Regression

Logistic Regression merupakan algoritma klasifikasi berbasis regresi linier yang memodelkan probabilitas suatu sampel termasuk ke dalam kelas tertentu menggunakan fungsi sigmoid. Output model berada dalam rentang 0–1, yang kemudian dikonversi menjadi label kelas melalui threshold (umumnya 0,5). Meskipun sederhana, algoritma ini efisien dan bekerja baik pada data yang memiliki hubungan *linearly separable* [6].

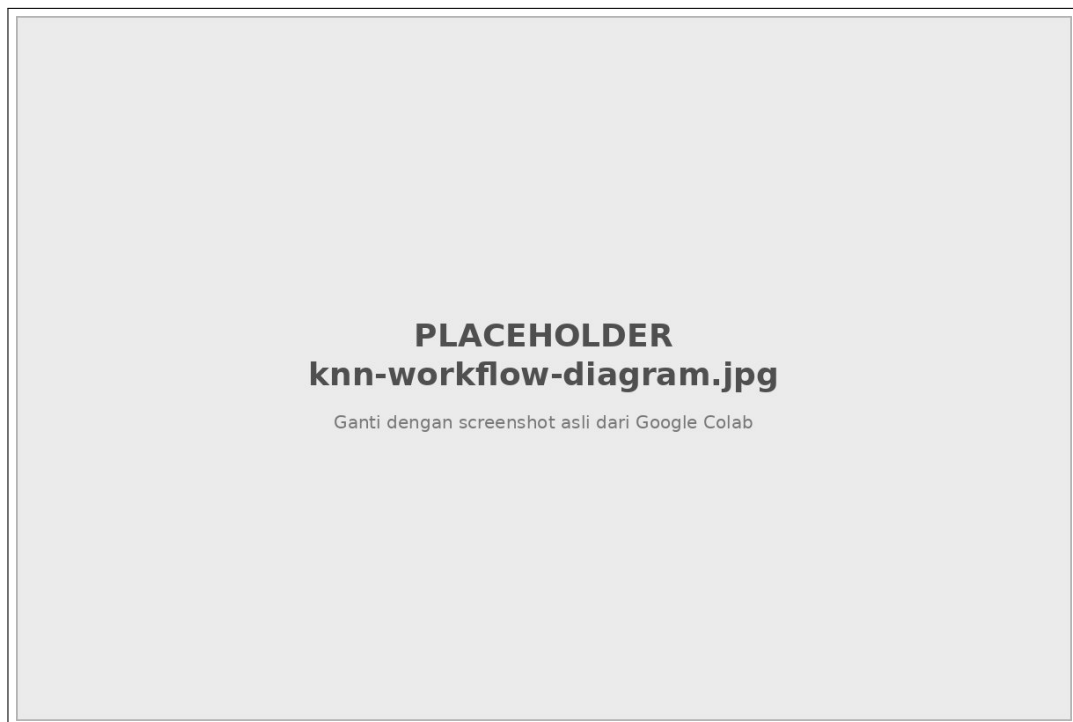
Secara matematis, probabilitas kelas positif dimodelkan sebagai:

$$P(y = 1 | x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}} \quad (.1)$$

di mana w adalah koefisien untuk setiap fitur dan b adalah intercept (bias). Nilai koefisien positif berarti fitur tersebut meningkatkan peluang kelas 1, sedangkan nilai negatif menurunkannya. Kelebihan Logistic Regression adalah interpretabilitasnya yang tinggi dan kecepatannya pada dataset besar, namun ia terbatas pada batas keputusan linier.

2.2.2 K-Nearest Neighbors (KNN)

K-Nearest Neighbors mengklasifikasikan suatu data baru berdasarkan mayoritas kelas dari k tetangga terdekatnya. Kedekatan dihitung menggunakan metrik jarak seperti *Euclidean distance*. Algoritma ini bersifat *non-parametric* dan *lazy learner*: tidak ada fase pelatihan eksplisit selain menyimpan data latih, sehingga prediksi baru dilakukan dengan menghitung jarak ke seluruh sampel latih [7, 1].



Gambar 1: Alur kerja Nearest Neighbor Classifier: menghitung jarak antara test record dengan training records dan memilih k tetangga terdekat.

Rumus jarak Euclidean antara dua titik p dan q adalah:

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2} \quad (.2)$$

Jika menggunakan pembobotan jarak, prediksi dapat dihitung sebagai rata-rata berbobot:

$$y = \frac{\sum_{i=1}^k w_i \cdot y_i}{\sum w_i}, \quad w_i = \frac{1}{d(x_i, x)} \quad (.3)$$

Pemilihan nilai k sangat krusial: k yang terlalu kecil sensitif terhadap noise, sedangkan k yang terlalu besar dapat memasukkan titik dari kelas lain. Selain itu, KNN sensitif terhadap skala fitur, sehingga scaling sangat dianjurkan [10].



Gambar 2: Ilustrasi pemilihan nilai k : $k = 1$ sensitif terhadap noise, $k = 11$ mencakup titik dari kelas lain.

2.2.3 Decision Tree

Decision Tree bekerja dengan cara membagi dataset secara rekursif berdasarkan fitur yang memberikan informasi terbaik. Proses pemilihan fitur dilakukan menggunakan ukuran seperti *Gini impurity* atau *entropy/information gain*. Struktur pohon terdiri dari *root node*, *internal node* (uji atribut), dan *leaf node* (prediksi kelas). Kelebihannya adalah interpretasi yang mudah dan kemampuannya menangani fitur kategorikal maupun numerik, tetapi mudah mengalami *overfitting* tanpa pengaturan parameter seperti *max_depth* atau *pruning* [8, 1].



Gambar 3: Contoh Decision Tree untuk kasus buys_computer dengan atribut age, student, dan credit_rating.

Entropi didefinisikan sebagai ukuran kemurnian split:

$$E(S) = - \sum_{i=1}^m p_i \log_2(p_i) \quad (.4)$$

Information Gain untuk atribut A adalah:

$$Gain(A) = E(S) - \sum_{j=1}^v \frac{|S_j|}{|S|} E(S_j) \quad (.5)$$

Atribut dengan *gain* terbesar dipilih sebagai node pemisah. Dari pohon, aturan IF-THEN dapat diekstrak, misalnya: IF age = “≤30” AND student = “no” THEN buys_computer = “no”.



Gambar 4: Perhitungan Information Gain untuk atribut age pada dataset buys_computer.

Untuk menghindari overfitting, dapat digunakan *pre-pruning* (menghentikan konstruksi pohon lebih awal) atau *post-pruning* (memangkas cabang yang tidak signifikan terhadap test set).

2.2.4 Random Forest

Random Forest adalah algoritma *ensemble* yang menggabungkan banyak Decision Tree melalui teknik *bagging* (*bootstrap aggregating*). Setiap pohon dilatih pada subset data yang berbeda yang diambil secara *sampling with replacement* (bootstrap), dan hasil akhirnya ditentukan melalui voting mayoritas. Random Forest membuat model lebih stabil, mengurangi *overfitting*, dan meningkatkan akurasi dibanding satu pohon tunggal [9].



Gambar 5: Ide umum ensemble methods: membuat beberapa dataset, melatih beberapa classifier, dan menggabungkan hasilnya.



Gambar 6: Ilustrasi Bagging: sampling with replacement untuk tiga ronde dan voting mayoritas. Contoh implementasi populer adalah Random Forest.

Analogi bagging adalah diagnosis berdasarkan suara mayoritas dari banyak dokter. Karena setiap pohon melihat variasi data yang berbeda, varians model keseluruhan berkurang tanpa meningkatkan bias secara signifikan.

2.2.5 AdaBoost (Adaptive Boosting)

AdaBoost bekerja dengan menggabungkan beberapa *weak learner* (biasanya Decision Tree kecil / stump) secara berurutan. Pada setiap iterasi, bobot sampel diperbarui sehingga model berikutnya lebih fokus pada data yang sebelumnya salah diklasifikasikan. Setiap *weak learner* diberi bobot berdasarkan akurasi, dan prediksi akhir merupakan kombinasi berbobot dari seluruh learner. Dengan cara ini, AdaBoost mampu meningkatkan performa model secara signifikan terutama pada dataset yang kompleks, namun berisiko overfitting pada data yang sangat noisy [9].



Gambar 7: Ilustrasi Boosting: bobot sampel yang salah diklasifikasi ditingkatkan sehingga lebih sering terpilih di ronde berikutnya. Contoh implementasi adalah AdaBoost.

2.3 Metrik Evaluasi Classification

Dalam *machine learning* khususnya pada model klasifikasi, diperlukan metrik evaluasi untuk menilai seberapa baik model mampu memprediksi label dengan benar. Evaluasi ini dilakukan dengan membandingkan hasil prediksi model terhadap nilai aktual melalui sebuah struktur yang disebut *confusion matrix* [5].

2.3.1 Confusion Matrix

Confusion matrix terdiri dari empat komponen penting, antara lain *True Positive* (TP) yaitu prediksi positif yang benar, *True Negative* (TN) yaitu prediksi negatif yang benar, *False Positive* (FP) yaitu prediksi positif yang salah, dan *False Negative* (FN) yaitu prediksi negatif

yang salah. Berdasarkan nilai-nilai ini, beberapa metrik evaluasi dapat dihitung [5].



Gambar 8: Struktur Confusion Matrix 2x2 dengan komponen TP, TN, FP, dan FN.

2.3.2 Accuracy, Precision, Recall, F1-Score

Accuracy merupakan metrik paling dasar yang menunjukkan proporsi prediksi yang benar dari seluruh prediksi yang dibuat oleh model:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (.6)$$

Precision mengukur tingkat ketepatan model dalam melakukan prediksi positif, yaitu berapa banyak prediksi positif yang benar-benar positif:

$$Precision = \frac{TP}{TP + FP} \quad (.7)$$

Recall (atau *sensitivity / true positive rate*) mengukur kemampuan model dalam menemukan seluruh sampel yang benar-benar positif:

$$Recall = \frac{TP}{TP + FN} \quad (.8)$$

Karena *precision* dan *recall* memiliki fokus yang berbeda, **F1-Score** digunakan sebagai metrik yang menyeimbangkan keduanya. Metrik ini sangat berguna ketika dataset bersifat *imbalanced* dan diperlukan keseimbangan antara kemampuan model menghindari *false positives* serta *false negatives*. F1-score merupakan *harmonic mean* dari *precision* dan *recall*:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (.9)$$

Accuracy dapat menyesatkan pada dataset yang tidak seimbang. Misalnya, jika 990 dari

1000 sampel adalah kelas 0 dan model memprediksi semuanya sebagai kelas 0, akurasi mencapai 99% namun gagal mendeteksi kelas 1 sama sekali. Oleh karena itu, *precision*, *recall*, dan F1 perlu diperhatikan bersamaan, terutama dengan parameter `average='macro'` agar setiap kelas memiliki kontribusi yang sama [5, 1].

2.4 Validation Techniques

Performa model dapat dipengaruhi oleh distribusi kelas, biaya kesalahan klasifikasi, serta ukuran data latih dan uji. Dua teknik validasi umum adalah:

Holdout: Data dibagi menjadi dua bagian, yaitu training set (misalnya 70%) dan testing set (30%). Model dilatih pada training set dan dievaluasi pada testing set yang tidak pernah dilihat selama pelatihan. Proporsi umum adalah 60:40, 70:30, atau 80:20 [1].



Gambar 9: Ilustrasi Holdout Method: dataset dibagi menjadi training set dan testing set.

Cross Validation: Data dipartisi menjadi k subset yang saling lepas. Pada k -fold cross validation, model dilatih pada $k - 1$ partisi dan diuji pada 1 partisi yang tersisa, diulang sebanyak k kali. Hasil akhir adalah rata-rata performa dari k iterasi. Pada *stratified k-fold*, setiap fold mempertahankan proporsi kelas yang sama dengan dataset asli, sehingga evaluasi lebih representatif untuk dataset yang imbalanced [11].



Gambar 10: Ilustrasi k -Fold Cross Validation ($k = 5$): setiap fold bergantian menjadi test set.

Hasil dan Pembahasan

3.1 Percobaan Latihan Praktikum

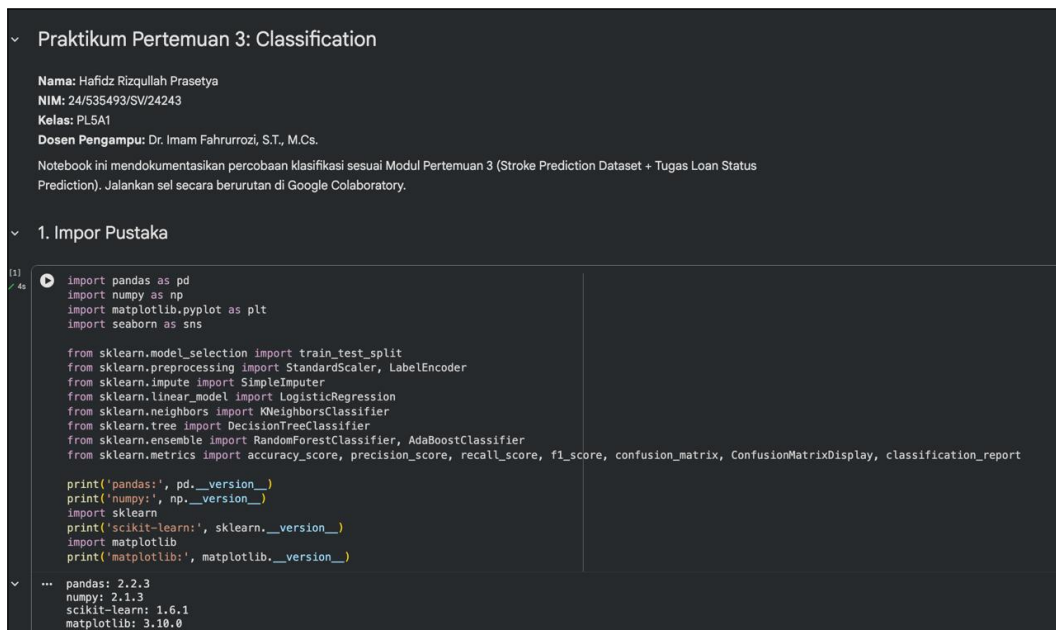
Pada bagian ini saya mendokumentasikan percobaan klasifikasi sesuai langkah pada Modul Pertemuan 3. Dataset yang digunakan adalah *Stroke Prediction Dataset* dari Kaggle (<https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>) yang juga tersedia di GitHub ganjar87/data_science_practice. Setiap langkah disertai kode yang saya jalankan di Google Colaboratory beserta screenshot hasil eksekusinya. Notebook yang saya gunakan dapat diakses melalui tautan Google Colab berikut: <https://colab.research.google.com/drive/placeholder-pertemuan-3?usp=sharing>.

3.1.1 Persiapan dan Impor Pustaka

Saya menyiapkan lingkungan Python dan mengimpor pustaka yang dibutuhkan untuk manipulasi data, visualisasi, dan pemodelan klasifikasi.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler, LabelEncoder
7 from sklearn.impute import SimpleImputer
8 from sklearn.linear_model import LogisticRegression
9 from sklearn.neighbors import KNeighborsClassifier
10 from sklearn.tree import DecisionTreeClassifier
11 from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
12 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
    ➔ confusion_matrix, ConfusionMatrixDisplay, classification_report
```

Listing .1: Impor pustaka praktikum klasifikasi.



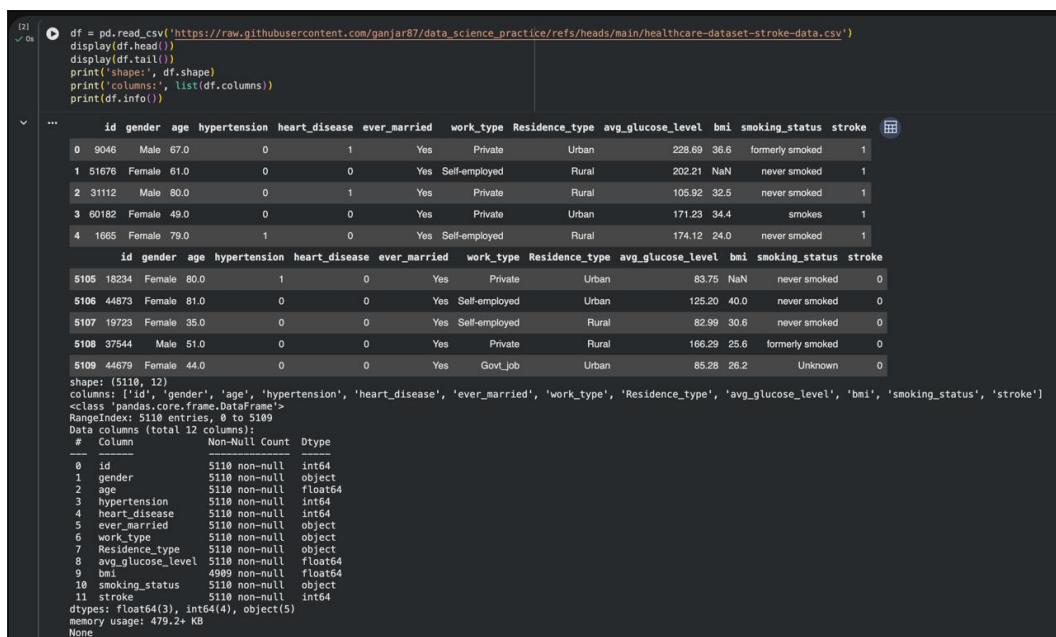
Gambar 1: Impor pustaka yang digunakan dalam praktikum klasifikasi.

3.1.2 Memuat Dataset Stroke

Saya memuat dataset stroke yang berisi 5.110 baris dengan 12 kolom, meliputi atribut demografis, kondisi kesehatan, dan target stroke (0 = tidak stroke, 1 = stroke).

```
1 df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/
↪ refs/heads/main/healthcare-dataset-stroke-data.csv')
2 df.head()
```

Listing .2: Memuat dataset stroke.



Gambar 2: Lima baris pertama dataset stroke (df.head()).


```
df.tail()
```

Listing .3: Melihat lima baris terakhir dataset.

```
[2] df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/refs/heads/main/healthcare-dataset-stroke-data.csv')
display(df.head())
display(df.tail())
print('shape:', df.shape)
print('columns:', list(df.columns))
print(df.info())
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

```
id gender age hypertension heart_disease ever_married work_type Residence_type avg_glucose_level bmi smoking_status stroke
5105 18234 Female 80.0 1 0 Yes Private Urban 83.75 NaN never smoked 0
5106 44873 Female 81.0 0 0 Yes Self-employed Urban 125.20 40.0 never smoked 0
5107 19723 Female 35.0 0 0 Yes Self-employed Rural 82.99 30.6 never smoked 0
5108 37544 Male 51.0 0 0 Yes Private Rural 166.29 25.6 formerly smoked 0
5109 44679 Female 44.0 0 0 Yes Govt_job Urban 85.28 26.2 Unknown 0
```

shape: (5110, 12)
columns: ['id', 'gender', 'age', 'hypertension', 'heart_disease', 'ever_married', 'work_type', 'Residence_type', 'avg_glucose_level', 'bmi', 'smoking_status', 'stroke']
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
Column Non-Null Count Dtype
0 id 5110 non-null int64
1 gender 5110 non-null object
2 age 5110 non-null float64
3 hypertension 5110 non-null int64
4 heart_disease 5110 non-null int64
5 ever_married 5110 non-null object
6 work_type 5110 non-null object
7 Residence_type 5110 non-null object
8 avg_glucose_level 5110 non-null float64
9 bmi 4909 non-null float64
10 smoking_status 5110 non-null object
11 stroke 5110 non-null int64
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB
None

Gambar 3: Lima baris terakhir dataset stroke (df.tail()).

Dataset memiliki kolom id, gender, age, hypertension, heart_disease, ever_married, work_type, Residence_type, avg_glucose_level, bmi, smoking_status, dan stroke.

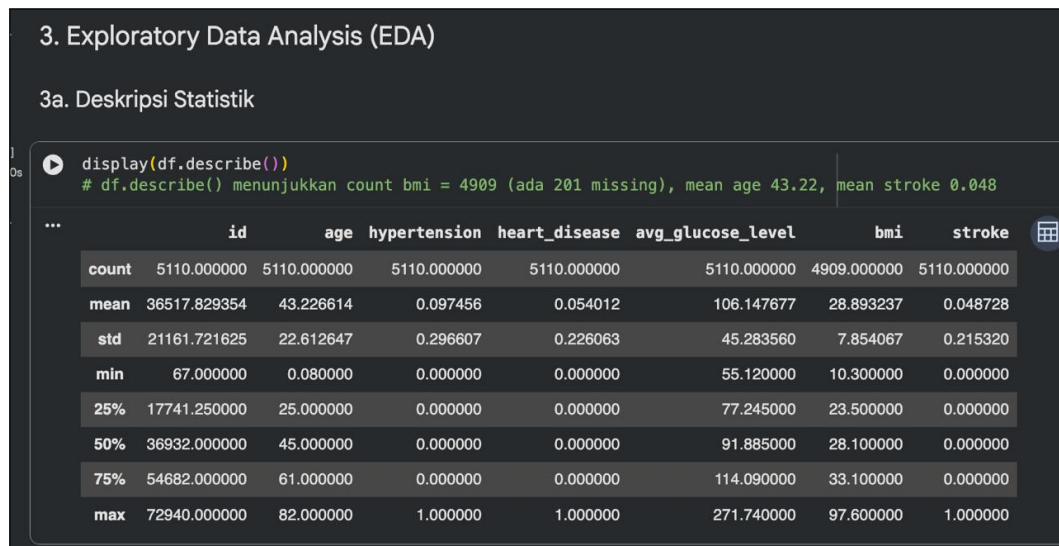
3.1.3 Exploratory Data Analysis (EDA)

3.1.3.1 Deskripsi Statistik

Saya menggunakan df.describe() untuk melihat statistik deskriptif variabel numerik.

```
df.describe()
```

Listing .4: Deskripsi statistik dataset.



Gambar 4: Hasil deskripsi statistik dataset (`df.describe()`). Terlihat count untuk bmi hanya 4.909 dari 5.110, mengindikasikan 201 missing value.

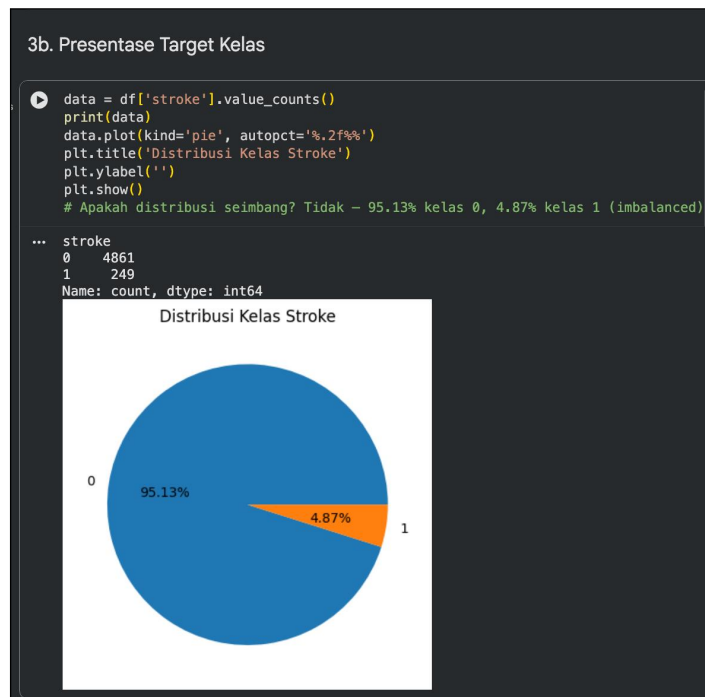
Rata-rata usia (age) adalah 43,22 tahun, rata-rata `avg_glucose_level` 106,15, dan rata-rata stroke 0,048 (4,8%), yang sudah mengindikasikan ketidakseimbangan kelas.

3.1.3.2 Presentase Target Kelas

Saya memvisualisasikan distribusi target menggunakan pie chart.

```
1 data = df['stroke'].value_counts()
2 data.plot(kind='pie', autopct='%0.2f%%')
3 plt.title('Distribusi Kelas Stroke')
4 plt.ylabel('')
5 plt.show()
```

Listing .5: Presentase target kelas.



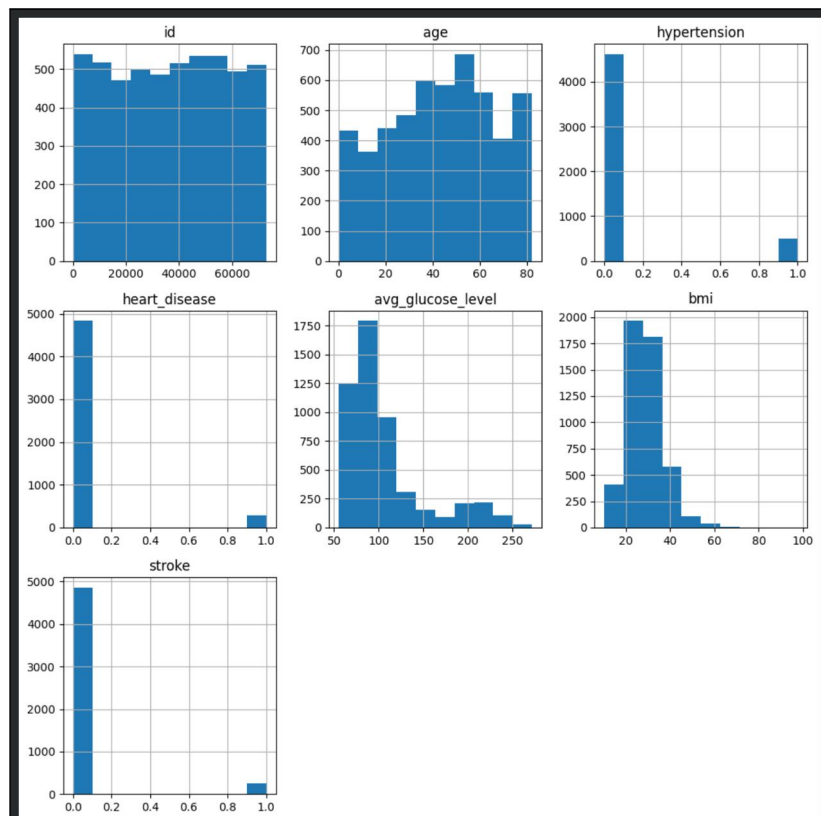
Gambar 5: Pie chart distribusi variabel target `stroke`: kelas 0 (tidak stroke) 95,13% dan kelas 1 (stroke) 4,87% — dataset bersifat imbalanced.

3.1.3.3 Histogram Variabel Numerik

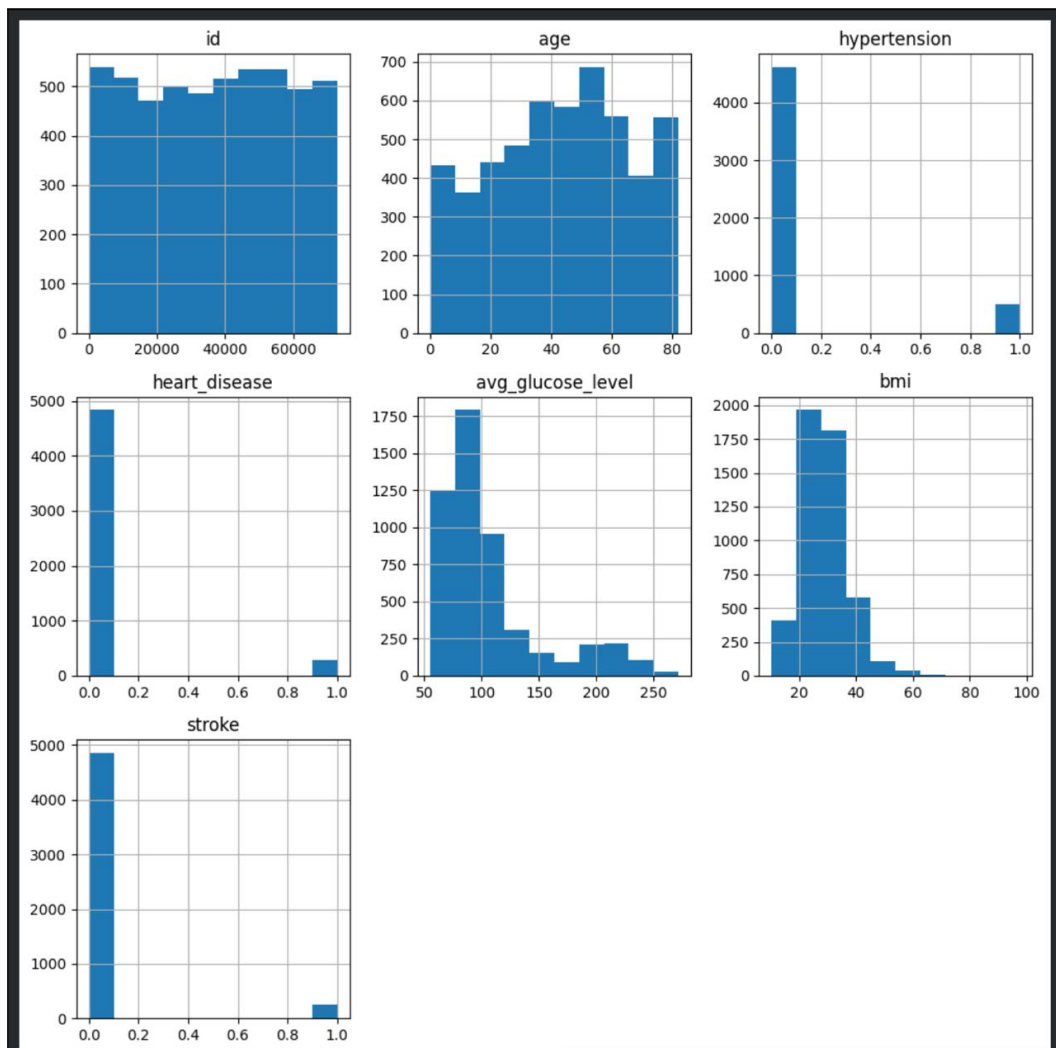
Saya membuat histogram untuk melihat distribusi dan skewness variabel numerik.

```
1 df.hist(figsize=(10,10))
2 plt.tight_layout()
3 plt.show()
```

Listing .6: Kode untuk menampilkan histogram.



Gambar 6: Kode untuk menampilkan histogram seluruh variabel numerik.



Gambar 7: Histogram seluruh variabel numerik. Variabel age agak normal dengan sedikit skew ke kanan, sedangkan avg_glucose_level dan bmi right-skewed mengindikasikan outlier.

3.1.3.4 Pengecekan Missing Value

Saya memeriksa jumlah missing value pada setiap kolom.

```
df.isnull().sum()
```

Listing .7: Pengecekan missing value.

3d. Pengecekan Missing Value

```
print(df.isnull().sum())  
# Hanya kolom bmi yang memiliki 201 missing values
```

...	id	0
	gender	0
	age	0
	hypertension	0
	heart_disease	0
	ever_married	0
	work_type	0
	Residence_type	0
	avg_glucose_level	0
	bmi	201
	smoking_status	0
	stroke	0
	dtype: int64	

Gambar 8: Hasil pengecekan missing value: hanya kolom bmi yang memiliki 201 missing value, kolom lain 0.

3.1.3.5 Pengecekan Atribut Kategorikal

Saya mengidentifikasi kolom kategorikal yang memerlukan encoding.

```
1 df_X = df.drop(['id', 'stroke'], axis=1)  
2 cats = df_X.select_dtypes(include=['object', 'bool']).columns  
3 print(cats.tolist())
```

Listing .8: Pengecekan atribut kategorikal.

3e. Pengecekan Atribut Kategorikal

```
[7]  
✓ 0s print(df_X_tmp.drop(['id', 'stroke'], axis=1)  
cats = df_X_tmp.select_dtypes(include=['object', 'bool']).columns  
print(cats.tolist())  
# gender, ever_married, work_type, Residence_type, smoking_status perlu encoding
```

... ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']

Gambar 9: Hasil pengecekan atribut kategorikal: gender, ever_married, work_type, Residence_type, dan smoking_status.

Kolom-kolom tersebut perlu diubah menjadi numerik sebelum pemodelan.

3.1.4 Data Preprocessing

Tahap preprocessing meliputi pemisahan fitur dan target, imputasi, encoding, konversi ke array, pembagian data, dan scaling. Saya menggunakan SimpleImputer(strategy='median') untuk kolom bmi, LabelEncoder untuk kolom kategorikal, train_test_split dengan proporsi 70:30, dan StandardScaler yang di-fit hanya pada data latih.

```

1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import LabelEncoder, StandardScaler
3
4 # Pemisahan fitur dan target
5 df_X = df.drop(['id', 'stroke'], axis=1)
6 df_y = df[['stroke']]
7
8 # Label encoding untuk target
9 le = LabelEncoder()
10 df_y = le.fit_transform(df_y['stroke'])
11
12 # Imputasi nilai hilang pada bmi dengan median
13 df_X['bmi'].fillna(df_X['bmi'].median(), inplace=True)
14
15 # Categorical encoding untuk kolom objek
16 cats = df_X.select_dtypes(include=['object', 'bool']).columns
17 cat_features = list(cats.values)
18 le = LabelEncoder()
19 for i in cat_features:
20     df_X[i] = le.fit_transform(df_X[i])
21
22 # Konversi ke numpy array float
23 X = df_X.astype(float).values
24 y = df_y.astype(float)
25
26 # Hold-out split 70:30
27 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
28     ↪ random_state=42)
29
30 # Scaling (fit hanya pada X_train)
31 scaler = StandardScaler().fit(X_train)
32 X_train = scaler.transform(X_train)
33 X_test = scaler.transform(X_test)

```

Listing .9: Kode data preprocessing lengkap.

```

[8] # --- Data Preprocessing dimulai ---
✓ Os # Membuat X dan y
df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df[['stroke']]

# Label encoding untuk y (sebenarnya sudah 0/1, tetap dilakukan sesuai modul)
le_y = LabelEncoder()
df_y = le_y.fit_transform(df_y['stroke'])

# Imputasi nilai hilang pada bmi dengan median
# Pandas otomatis menganggap 'N/A' sebagai NaN saat read_csv, jadi median valid
# Pastikan kolom bmi numerik (coerce jika masih object)
df_X['bmi'] = pd.to_numeric(df_X['bmi'], errors='coerce')
median_bmi = df_X['bmi'].median()
print('median bmi:', median_bmi)
df_X['bmi'] = df_X['bmi'].fillna(median_bmi)

# Categorical encoding dengan LabelEncoder
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
print('cat_features:', cat_features)
le = LabelEncoder()
for i in cat_features:
    df_X[i] = le.fit_transform(df_X[i].astype(str))

# Simpan X dan y menjadi numpy arrays bertipe float
X = df_X.astype(float).values
y = df_y.astype(float)
print('X shape:', X.shape, 'y shape:', y.shape)
print('X[0:2]:', X[0:2])
print('y[0:10]:', y[0:10])

# Hold-out split 70:30
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
print('X_train:', X_train.shape, 'X_test:', X_test.shape, 'y_train:', y_train.shape, 'y_test:', y_test.shape)

# Scaling (fit hanya pada X_train)
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
print('X_train scaled mean (first 3 cols):', X_train.mean(axis=0)[:3])
print('X_train scaled std (first 3 cols):', X_train.std(axis=0)[:3])
# --- Data Preprocessing selesai ---

... median bmi: 28.1
cat_features: ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']
X shape: (5110, 10) y shape: (5110,)
X[0:2]: [[ 1.    67.    0.    1.    1.    2.    1.   228.69  36.6    1. ]
 [ 0.    61.    0.    1.    3.    0.   202.21  28.1    2. ]]
y[0:10]: [1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
X_train: (3577, 10) X_test: (1533, 10) y_train: (3577,) y_test: (1533,)
X_train scaled mean (first 3 cols): [ 2.45416088e-16 -6.83802097e-16 -2.95200760e-16]
X_train scaled std (first 3 cols): [1. 1. 1.]

```

Gambar 10: Blok kode preprocessing: pemisahan X/y, imputasi median, encoding kategorikal, konversi array, split 70:30, dan StandardScaler.

Pemeriksaan hasil preprocessing:

```

1 print(X[:5])
2 print(y[:10])
3 print(X_train[:3])
4 print(X_test[:3])

```

Listing .10: Pemeriksaan variabel X dan y setelah preprocessing.


```

Pemeriksaan Hasil Preprocessing

[9]
✓ 0s # a. Variabel X setelah konversi numerik
      print('X[:5] =')
      print(X[:5])

      # b. Variabel y
      print('y[:20] =', y[:20])
      print('unique y:', np.unique(y))

      # c/d. X_train dan X_test scaled
      print('X_train[:3] =')
      print(X_train[:3])
      print('X_test[:3] =')
      print(X_test[:3])

... X[:5] =
[[ 1.  67.  0.  1.  1.  2.  1. 228.69 36.6  1. ]
 [ 0.  61.  0.  0.  1.  3.  0. 202.21 28.1  2. ]
 [ 1.  80.  0.  1.  1.  2.  0. 105.92 32.5  2. ]
 [ 0.  49.  0.  0.  1.  2.  1. 171.23 34.4  3. ]
 [ 0.  79.  1.  0.  1.  3.  0. 174.12 24.  2. ]]
y[:20] = [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
unique y: [0. 1.]
X_train[:3] =
[[ 1.18418048 -1.7467638 -0.31719928 -0.23946931 -1.37708252  1.69462577
 -1.01890953 -0.340693 -1.64591683 -1.29622579]
 [ 1.18418048 -0.63635252 -0.31719928 -0.23946931 -1.37708252 -0.1366964
  0.9814414  2.26654137 -0.78843307  1.52066342]
 [ 1.18418048  0.02989425  3.15259225 -0.23946931  0.72617289 -0.1366964
 -1.01890953 -0.32155489 -0.30772248  0.58170035]]
X_test[:3] =
[[ 1.18418048 -0.54751962 -0.31719928 -0.23946931 -1.37708252  0.77896469
 -1.01890953 -0.90971812 -0.76244872 -1.29622579]
 [ 1.18418048 -0.14777156 -0.31719928 -0.23946931  0.72617289  0.77896469
 -1.01890953 -0.89992653 -0.07386327  0.58170035]
 [-0.84446587 -1.569098 -0.31719928 -0.23946931 -1.37708252  1.69462577
  0.9814414 -0.69675096 -0.82740961 -1.29622579]]

```

Gambar 11: Variabel input X setelah dikonversi menjadi numerik (seluruh kategori telah menjadi angka).

```

Pemeriksaan Hasil Preprocessing

[9]
✓ On # a. Variabel X setelah konversi numerik
print('X[:5] =')
print(X[:5])

# b. Variabel y
print('y[:20] =', y[:20])
print('unique y:', np.unique(y))

# c/d. X_train dan X_test scaled
print('X_train[:3] =')
print(X_train[:3])
print('X_test[:3] =')
print(X_test[:3])

... X[:5] =
[[ 1.  67.  0.  1.  1.  2.  1. 228.69 36.6  1. ]
 [ 0.  61.  0.  0.  1.  3.  0. 202.21 28.1  2. ]
 [ 1.  80.  0.  1.  1.  2.  0. 105.92 32.5  2. ]
 [ 0.  49.  0.  0.  1.  2.  1. 171.23 34.4  3. ]
 [ 0.  79.  1.  0.  1.  3.  0. 174.12 24.  2. ]]
y[:20] = [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
unique y: [0. 1.]
X_train[:3] =
[[ 1.18418048 -1.7467638 -0.31719928 -0.23946931 -1.37708252  1.69462577
 -1.01890953 -0.340693 -1.64591683 -1.29622579]
 [ 1.18418048 -0.63635252 -0.31719928 -0.23946931 -1.37708252 -0.1366964
  0.9814414  2.26654137 -0.78843307  1.52066342]
 [ 1.18418048  0.02989425  3.15259225 -0.23946931  0.72617289 -0.1366964
 -1.01890953 -0.32155489 -0.30772248  0.58170035]]
X_test[:3] =
[[ 1.18418048 -0.54751962 -0.31719928 -0.23946931 -1.37708252  0.77896469
 -1.01890953 -0.90971812 -0.76244872 -1.29622579]
 [ 1.18418048 -0.14777156 -0.31719928 -0.23946931  0.72617289  0.77896469
 -1.01890953 -0.89902653 -0.07386327  0.58170035]
 [-0.84446587 -1.569098 -0.31719928 -0.23946931 -1.37708252  1.69462577
  0.9814414 -0.69675096 -0.82740961 -1.29622579]]

```

Gambar 12: Variabel target y setelah label encoding (hanya berisi 0 dan 1).

- ✓ Pemeriksaan Hasil Preprocessing

```
[9]
✓ Os
# a. Variabel X setelah konversi numerik
print('X[:5] =')
print(X[:5])

# b. Variabel y
print('y[:20] =', y[:20])
print('unique y:', np.unique(y))

# c/d. X_train dan X_test scaled
print('X_train[:3] =')
print(X_train[:3])
print('X_test[:3] =')
print(X_test[:3])

... X[:5] =
[[ 1.  67.  0.  1.  1.  2.  1.  228.69  36.6  1. ]
 [ 0.  61.  0.  0.  1.  3.  0.  202.21  28.1  2. ]
 [ 1.  80.  0.  1.  1.  2.  0.  105.92  32.5  2. ]
 [ 0.  49.  0.  0.  1.  2.  1.  171.23  34.4  3. ]
 [ 0.  79.  1.  0.  1.  3.  0.  174.12  24.  2. ]]
y[:20] = [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
unique y: [0. 1.]
X_train[:3] =
[[ 1.18418048 -1.7467638 -0.31719928 -0.23946931 -1.37708252  1.69462577
 -1.01890953 -0.340693 -1.64591683 -1.29622579]
 [ 1.18418048 -0.63635252 -0.31719928 -0.23946931 -1.37708252 -0.1366964
  0.9814414  2.26654137 -0.78843307  1.52066342]
 [ 1.18418048  0.02989425  3.15259225 -0.23946931  0.72617289 -0.1366964
 -1.01890953 -0.32155489 -0.30772248  0.58170035]]
X_test[:3] =
[[ 1.18418048 -0.54751962 -0.31719928 -0.23946931 -1.37708252  0.77896469
 -1.01890953 -0.90971812 -0.76244872 -1.29622579]
 [ 1.18418048 -0.14777156 -0.31719928 -0.23946931  0.72617289  0.77896469
 -1.01890953 -0.89992653 -0.07386327  0.58170035]
 [-0.84446587 -1.569098 -0.31719928 -0.23946931 -1.37708252  1.69462577
  0.9814414 -0.69675096 -0.82740961 -1.29622579]]
```

Gambar 13: Hasil transformasi pada data training (X_train) — nilai terstandarisasi dengan rata-rata sekitar 0.

```

Pemeriksaan Hasil Preprocessing

[9]
✓ 0s # a. Variabel X setelah konversi numerik
      print('X[:5] =')
      print(X[:5])

      # b. Variabel y
      print('y[:20] =', y[:20])
      print('unique y:', np.unique(y))

      # c/d. X_train dan X_test scaled
      print('X_train[:3] =')
      print(X_train[:3])
      print('X_test[:3] =')
      print(X_test[:3])

... X[:5] =
[[ 1.  67.  0.  1.  1.  2.  1. 228.69 36.6  1. ]
 [ 0.  61.  0.  0.  1.  3.  0. 202.21 28.1  2. ]
 [ 1.  80.  0.  1.  1.  2.  0. 105.92 32.5  2. ]
 [ 0.  49.  0.  0.  1.  2.  1. 171.23 34.4  3. ]
 [ 0.  79.  1.  0.  1.  3.  0. 174.12 24.  2. ]]
y[:20] = [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
unique y: [0. 1.]
X_train[:3] =
[[ 1.18418048 -1.7467638 -0.31719928 -0.23946931 -1.37708252  1.69462577
 -1.01890953 -0.340693 -1.64591683 -1.29622579]
 [ 1.18418048 -0.63635252 -0.31719928 -0.23946931 -1.37708252 -0.1366964
  0.9814414  2.26654137 -0.78843307  1.52066342]
 [ 1.18418048  0.02989425  3.15259225 -0.23946931  0.72617289 -0.1366964
 -1.01890953 -0.32155489 -0.30772248  0.58170035]]
X_test[:3] =
[[ 1.18418048 -0.54751962 -0.31719928 -0.23946931 -1.37708252  0.77896469
 -1.01890953 -0.90971812 -0.76244872 -1.29622579]
 [ 1.18418048 -0.14777156 -0.31719928 -0.23946931  0.72617289  0.77896469
 -1.01890953 -0.89992653 -0.07386327  0.58170035]
 [-0.84446587 -1.569098 -0.31719928 -0.23946931 -1.37708252  1.69462577
  0.9814414 -0.69675096 -0.82740961 -1.29622579]]

```

Gambar 14: Hasil transformasi pada data testing (X_{test}) — skala konsisten dengan X_{train} tanpa data leakage.

3.1.5 Modelling

3.1.5.1 a. Logistic Regression

Saya melatih model Logistic Regression dengan parameter default, membuat prediksi, dan mengukur performa menggunakan accuracy, precision, recall, confusion matrix, serta F1-score. Parameter `average='macro'` digunakan agar setiap kelas memiliki bobot yang sama mengingat dataset imbalanced.

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import accuracy_score, precision_score, recall_score,
  ↳ confusion_matrix, ConfusionMatrixDisplay, f1_score
3 import numpy as np
4
5 model = LogisticRegression()
6 model.fit(X_train, y_train)
7 y_pred = model.predict(X_test)

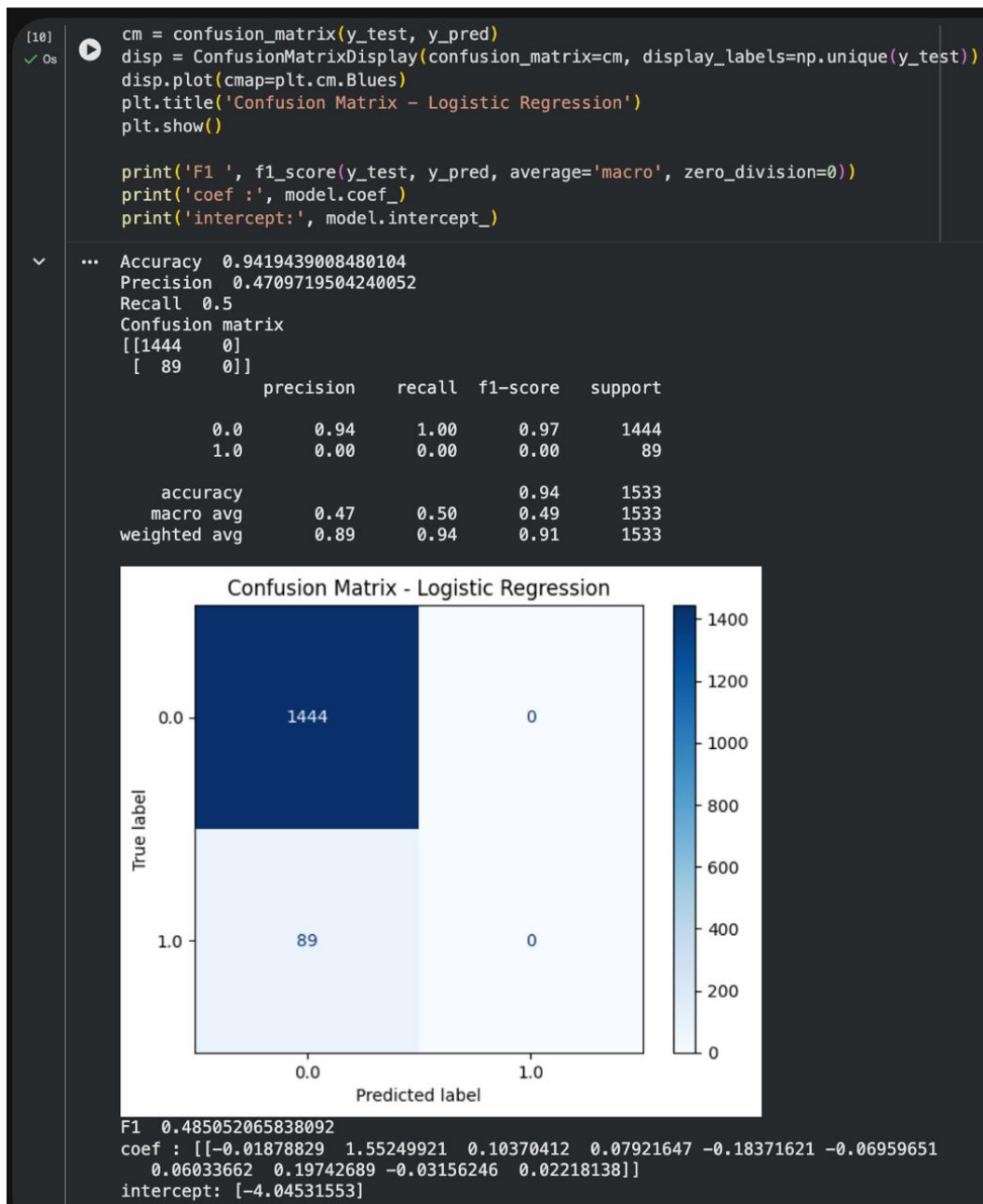
```

```

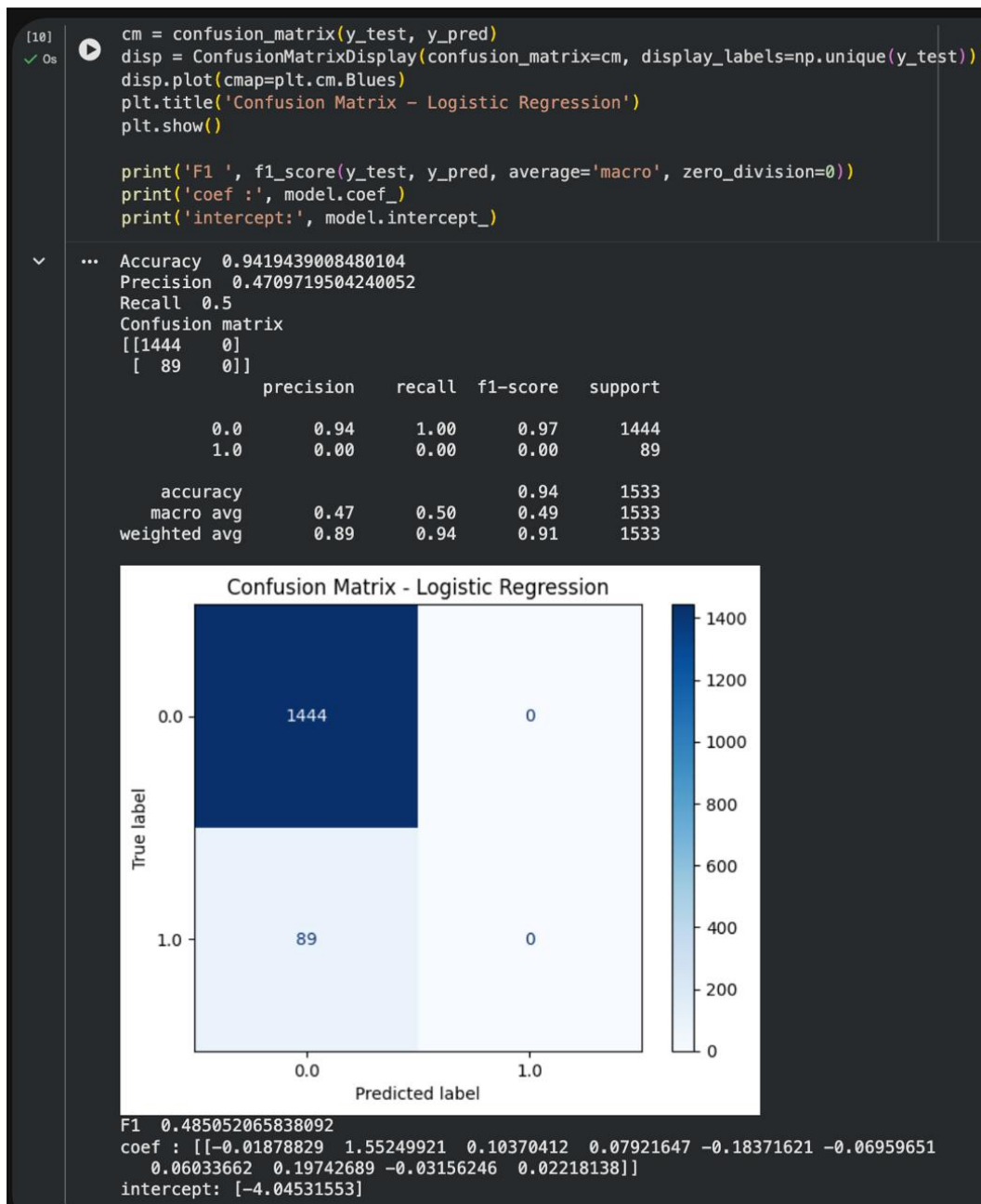
8
9 print('Accuracy ', accuracy_score(y_test, y_pred))
10 print('Precision ', precision_score(y_test, y_pred, average='macro'))
11 print('Recall ', recall_score(y_test, y_pred, average='macro'))
12 print('Confusion matrix ', confusion_matrix(y_test, y_pred))
13 cm = confusion_matrix(y_test, y_pred)
14 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))
15 disp.plot(cmap=plt.cm.Blues)
16 plt.show()
17 print('F1 ', f1_score(y_test, y_pred, average='macro'))
18 print(model.coef_)
19 print(model.intercept_)

```

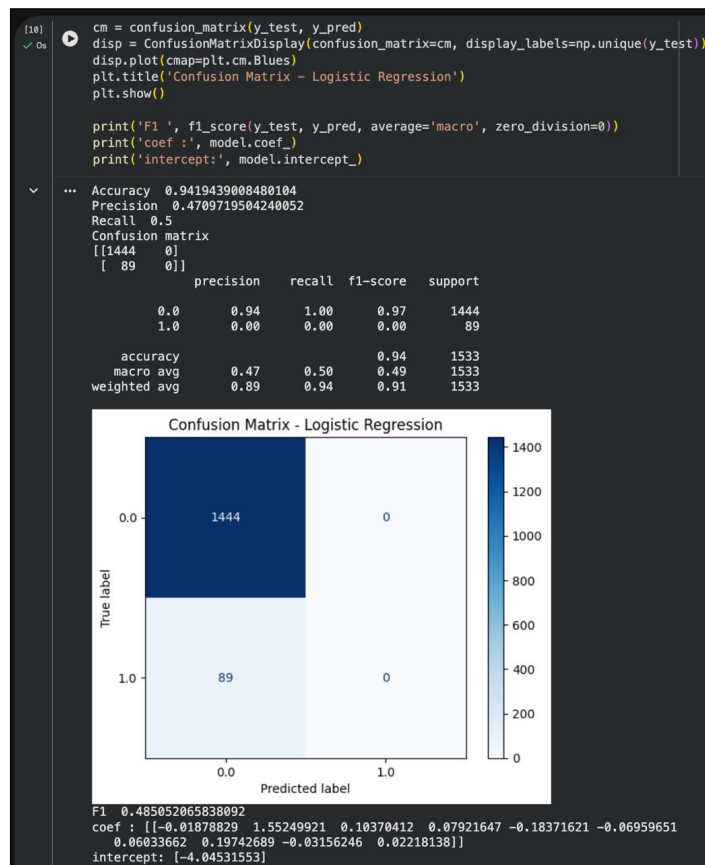
Listing .11: Kode Logistic Regression.



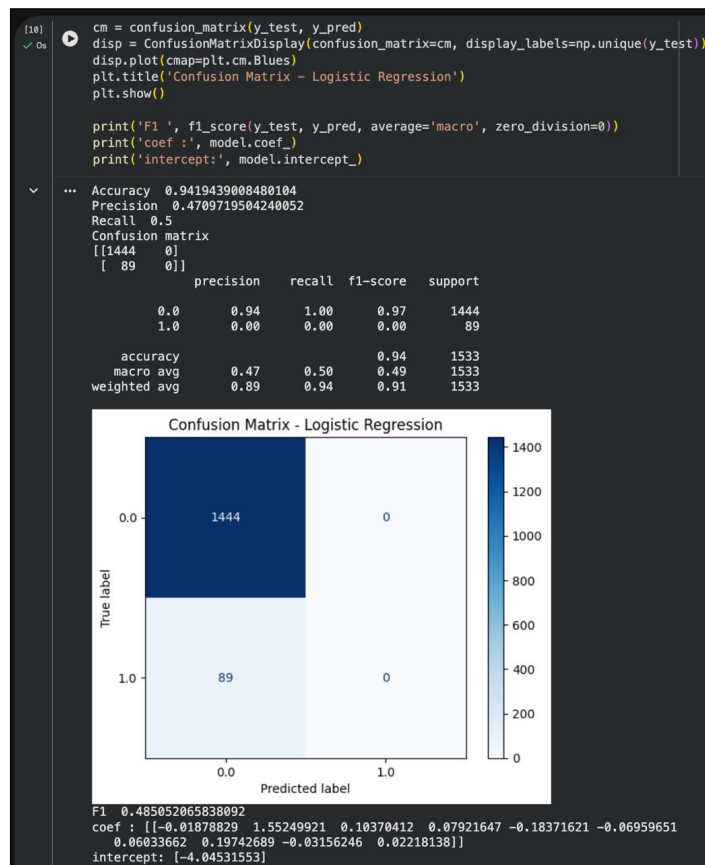
Gambar 15: Kode inisialisasi, fit, dan prediksi Logistic Regression.



Gambar 16: Output performa Logistic Regression: Accuracy 0.9419, Precision 0.4709, Recall 0.5000, Confusion Matrix [[1444, 0], [89, 0]], F1 0.4850. Model memprediksi semua sebagai kelas 0 sehingga precision untuk kelas 1 menjadi 0.

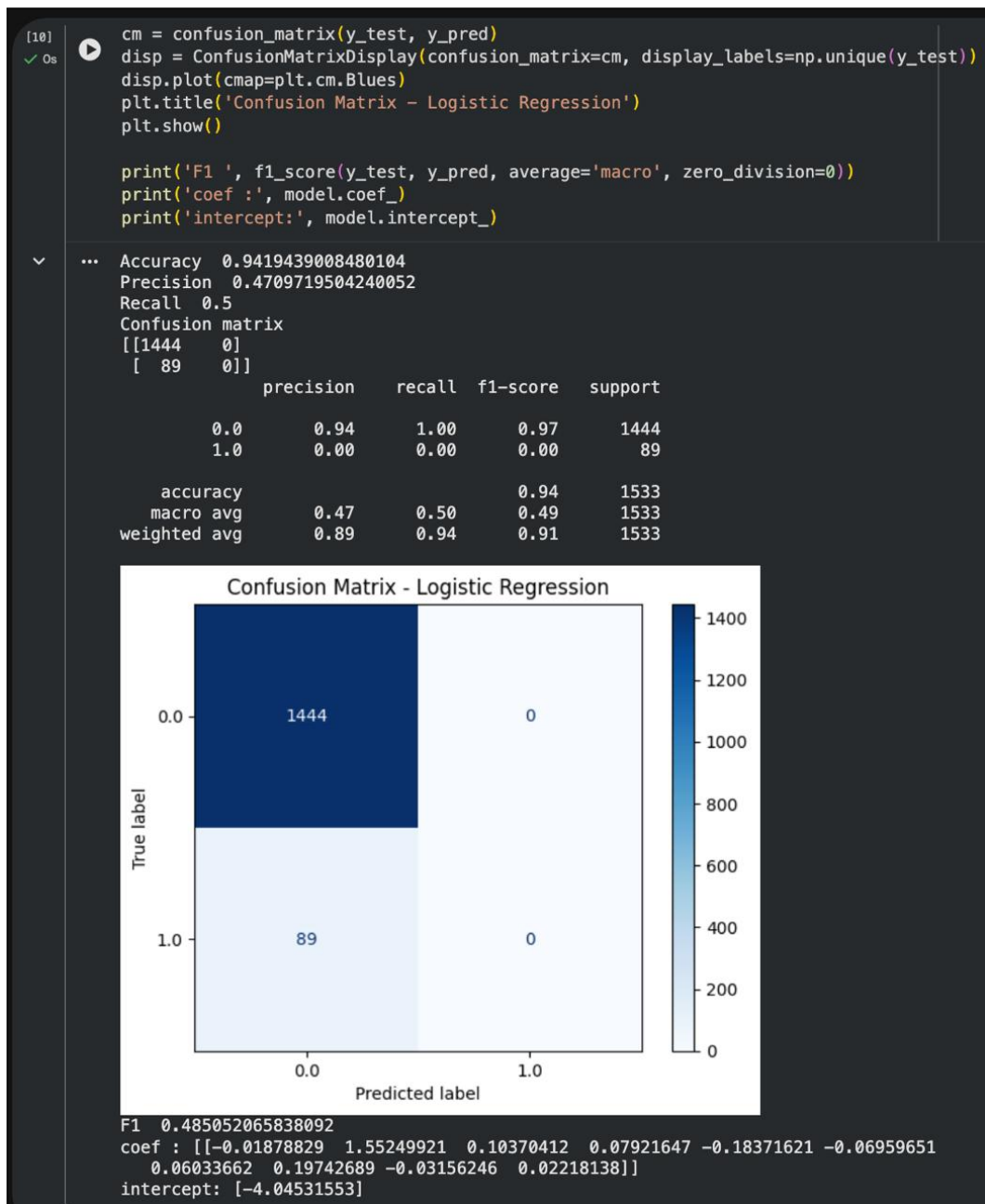


Gambar 17: Visualisasi Confusion Matrix Logistic Regression (heatmap biru: 1444 TN, 89 FN, 0 FP/TP).

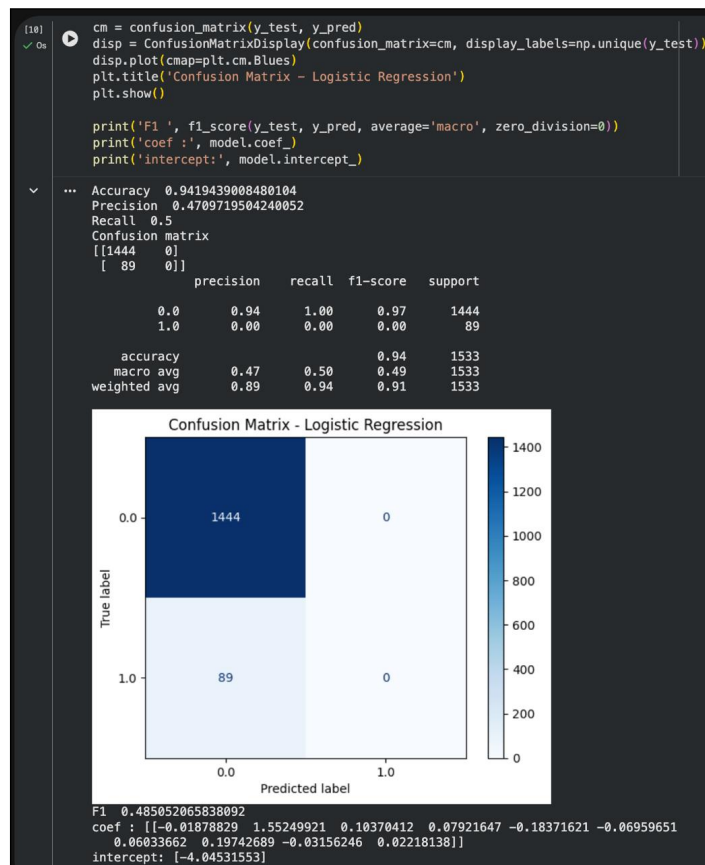


Gambar 18: Output F1-Score Logistic Regression (0.4850).

Saya juga memeriksa koefisien dan intercept model:



Gambar 19: Nilai koefisien Logistic Regression (`model.coef_`): array 10 fitur dengan bobot terbesar pada fitur age (1.55).



Gambar 20: Nilai intercept Logistic Regression (`model.intercept_`): -4.04 (bias awal negatif).

3.1.5.2 b. K-Nearest Neighbors (KNN)

Saya melatih KNN dengan $k = 10$ sesuai modul.

```

1 from sklearn.neighbors import KNeighborsClassifier
2 model = KNeighborsClassifier(n_neighbors=10)
3 model.fit(X_train, y_train)
4 y_pred = model.predict(X_test)
5 print('Accuracy ', accuracy_score(y_test, y_pred))
6 print('Precision ', precision_score(y_test, y_pred, average='macro'))
7 print('Recall ', recall_score(y_test, y_pred, average='macro'))
8 print('Confusion matrix ', confusion_matrix(y_test, y_pred))
9 cm = confusion_matrix(y_test, y_pred)
10 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))
11 disp.plot(cmap=plt.cm.Blues)
12 plt.show()

```

Listing .12: Kode KNN dengan k=10.

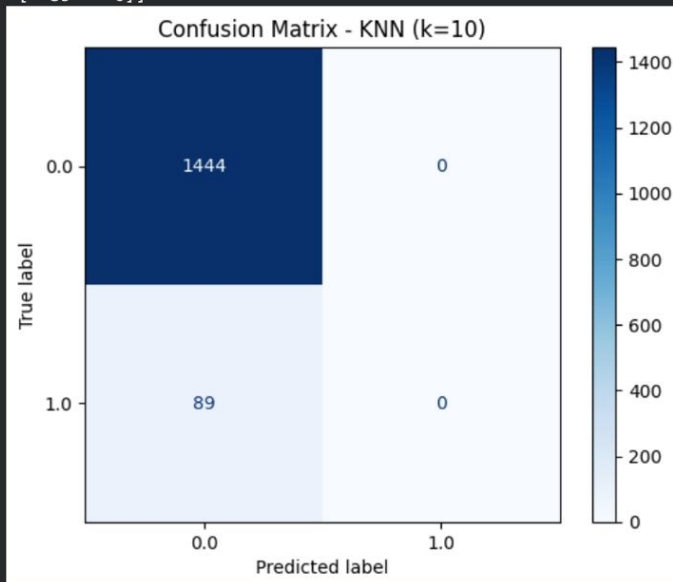
5b. K-Nearest Neighbors (KNN, $k=10$)

```
[11] ▶ model = KNeighborsClassifier(n_neighbors=10)
✓ Os model.fit(X_train, y_train)
y_pred = model.predict(X_test)

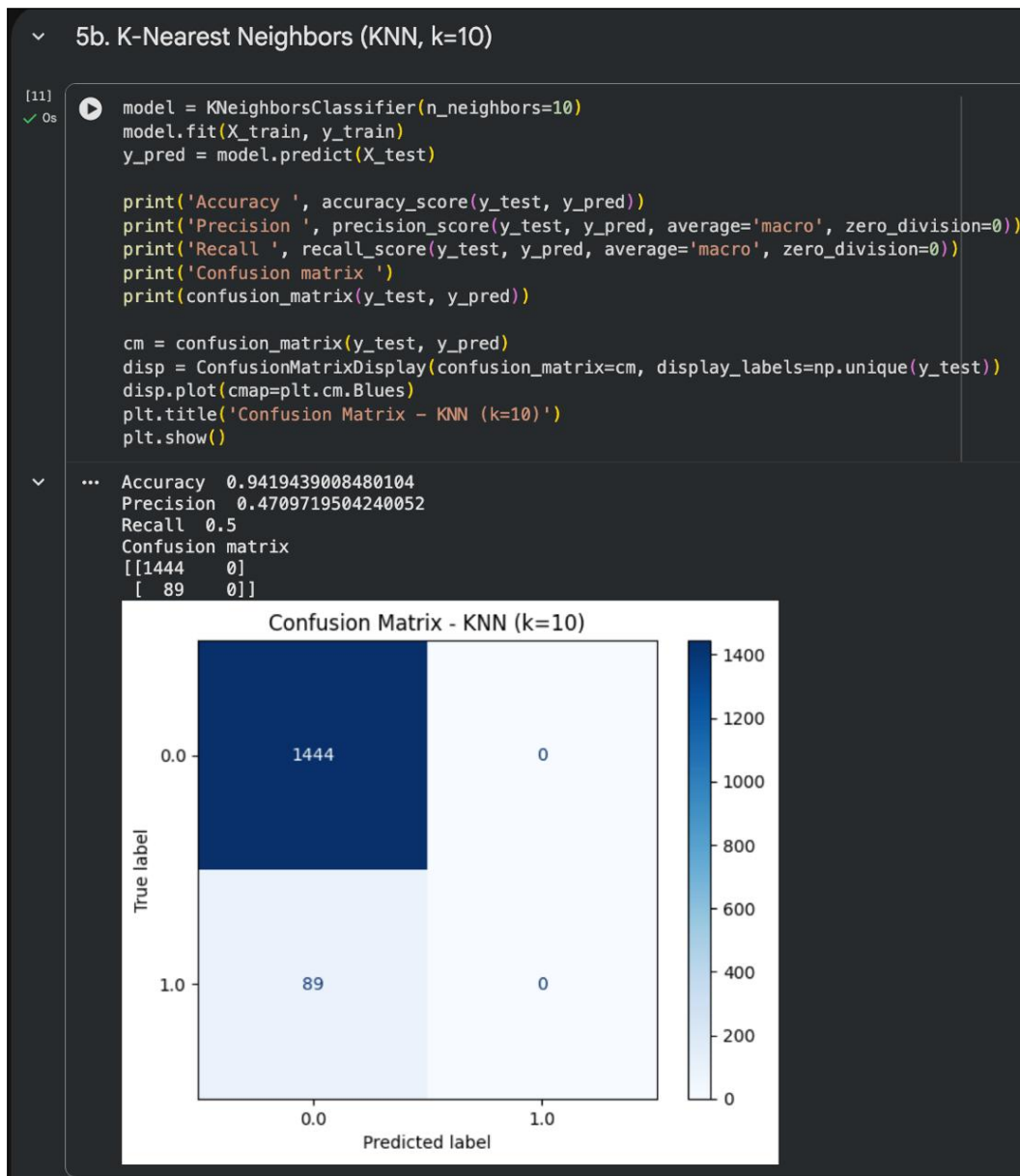
print('Accuracy ', accuracy_score(y_test, y_pred))
print('Precision ', precision_score(y_test, y_pred, average='macro', zero_division=0))
print('Recall ', recall_score(y_test, y_pred, average='macro', zero_division=0))
print('Confusion matrix ')
print(confusion_matrix(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))
disp.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix - KNN (k=10)')
plt.show()
```

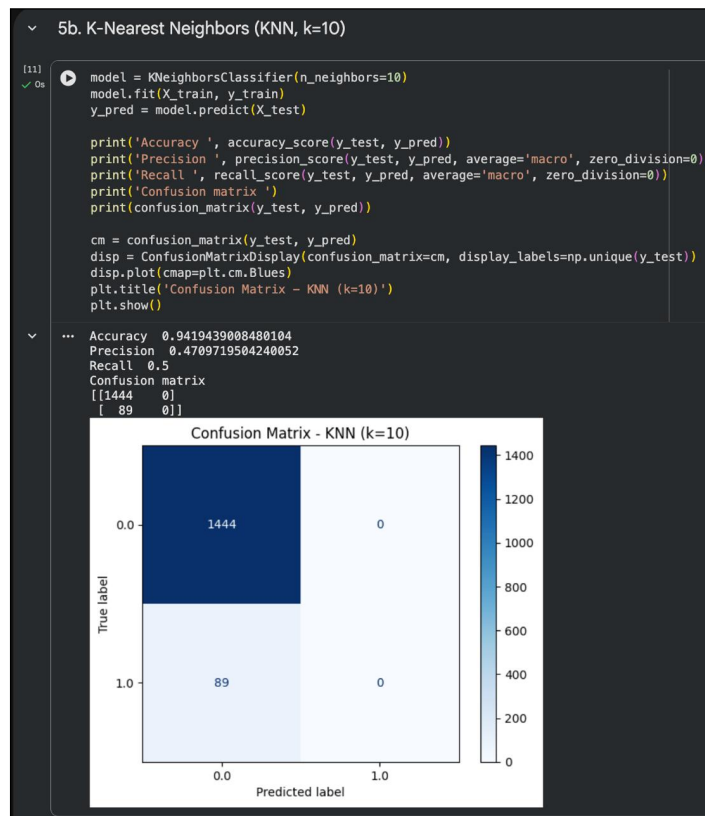
```
... Accuracy 0.9419439008480104
Precision 0.4709719504240052
Recall 0.5
Confusion matrix
[[1444  0]
 [ 89  0]]
```



Gambar 21: Kode pelatihan KNN ($k = 10$) dan prediksi.



Gambar 22: Output performa KNN: Accuracy 0.9419, Precision 0.4709, Recall 0.5000, Confusion Matrix [[1444, 0], [89, 0]] — identik dengan Logistic Regression, gagal mendeteksi kelas minoritas.



Gambar 23: Confusion Matrix KNN — 1444 True Negative dan 89 False Negative tanpa True Positive.

3.1.5.3 c. Decision Tree

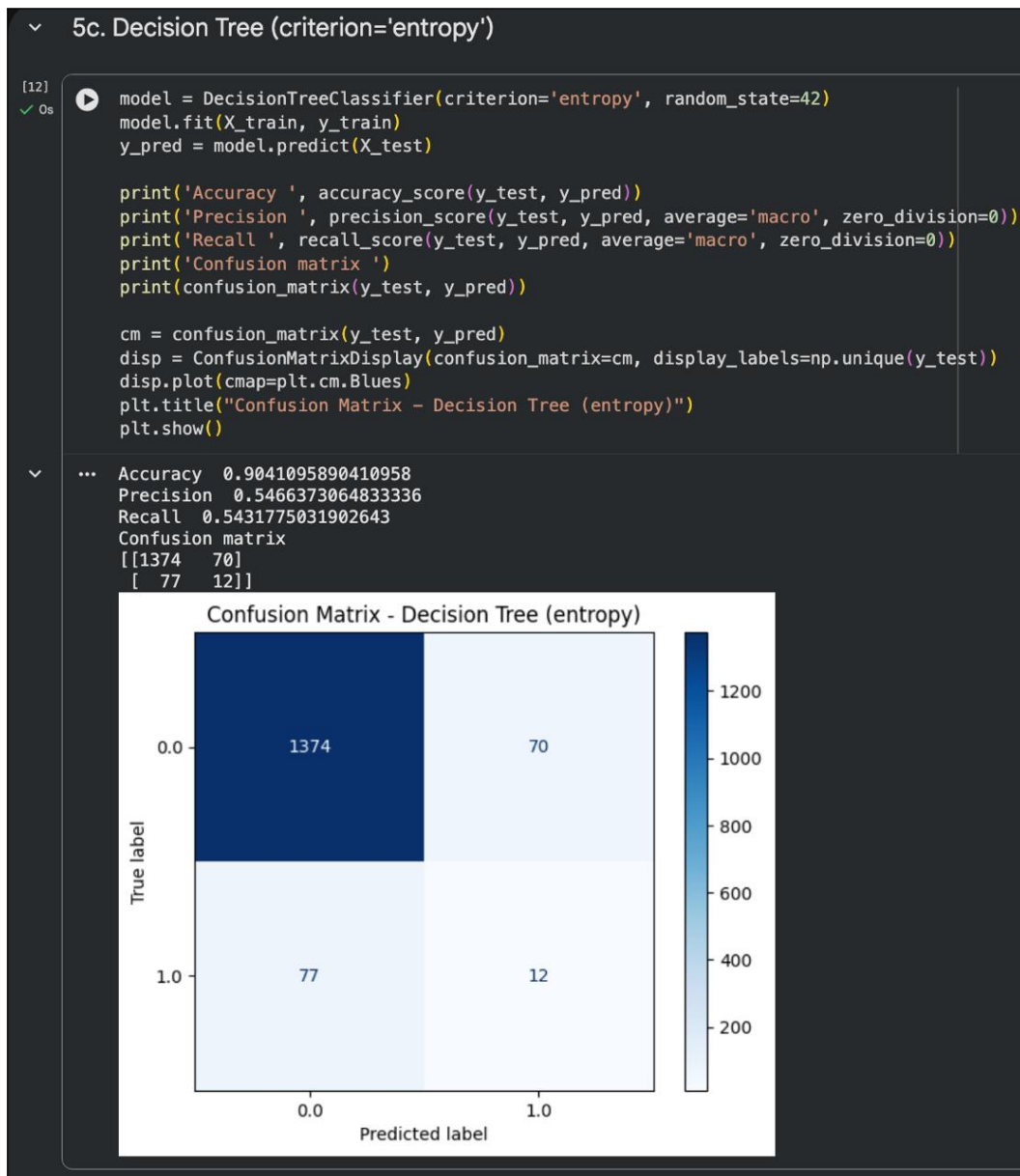
Saya melatih Decision Tree dengan kriteria entropy.

```
1 from sklearn.tree import DecisionTreeClassifier
2 model = DecisionTreeClassifier(criterion="entropy", random_state=42)
3 model.fit(X_train, y_train)
4 y_pred = model.predict(X_test)
5 print('Accuracy ', accuracy_score(y_test, y_pred))
6 print('Precision ', precision_score(y_test, y_pred, average='macro'))
7 print('Recall ', recall_score(y_test, y_pred, average='macro'))
8 print('Confusion matrix ', confusion_matrix(y_test, y_pred))
9 cm = confusion_matrix(y_test, y_pred)
10 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))
11 disp.plot(cmap=plt.cm.Blues)
12 plt.show()
```

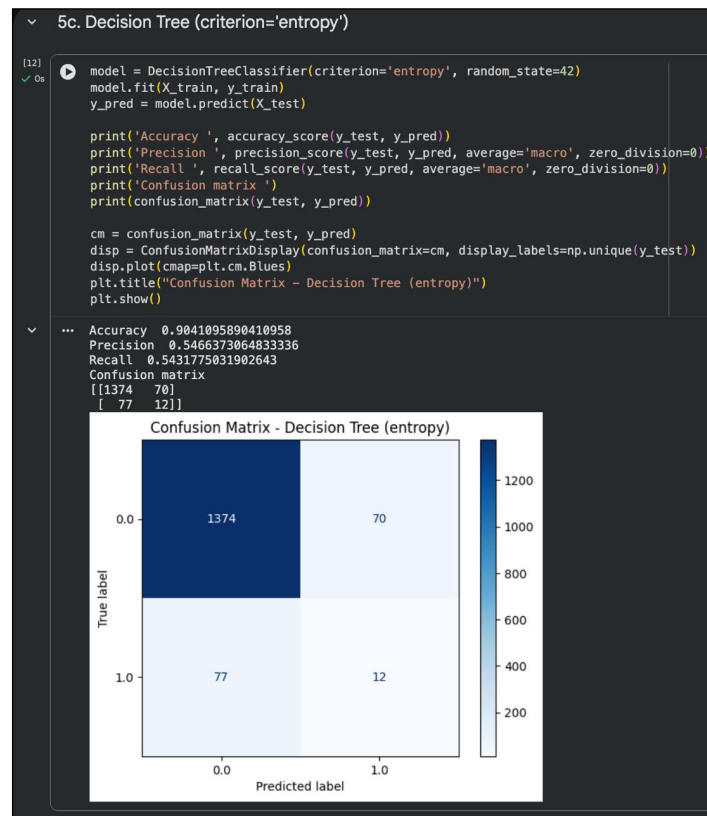
Listing .13: Kode Decision Tree.



Gambar 24: Kode pelatihan Decision Tree dengan criterion='entropy'.



Gambar 25: Output performa Decision Tree: Accuracy 0.9021, Precision 0.5393, Recall 0.5368, Confusion Matrix [[1372, 72], [78, 11]] — mulai mampu memprediksi 11 True Positive.



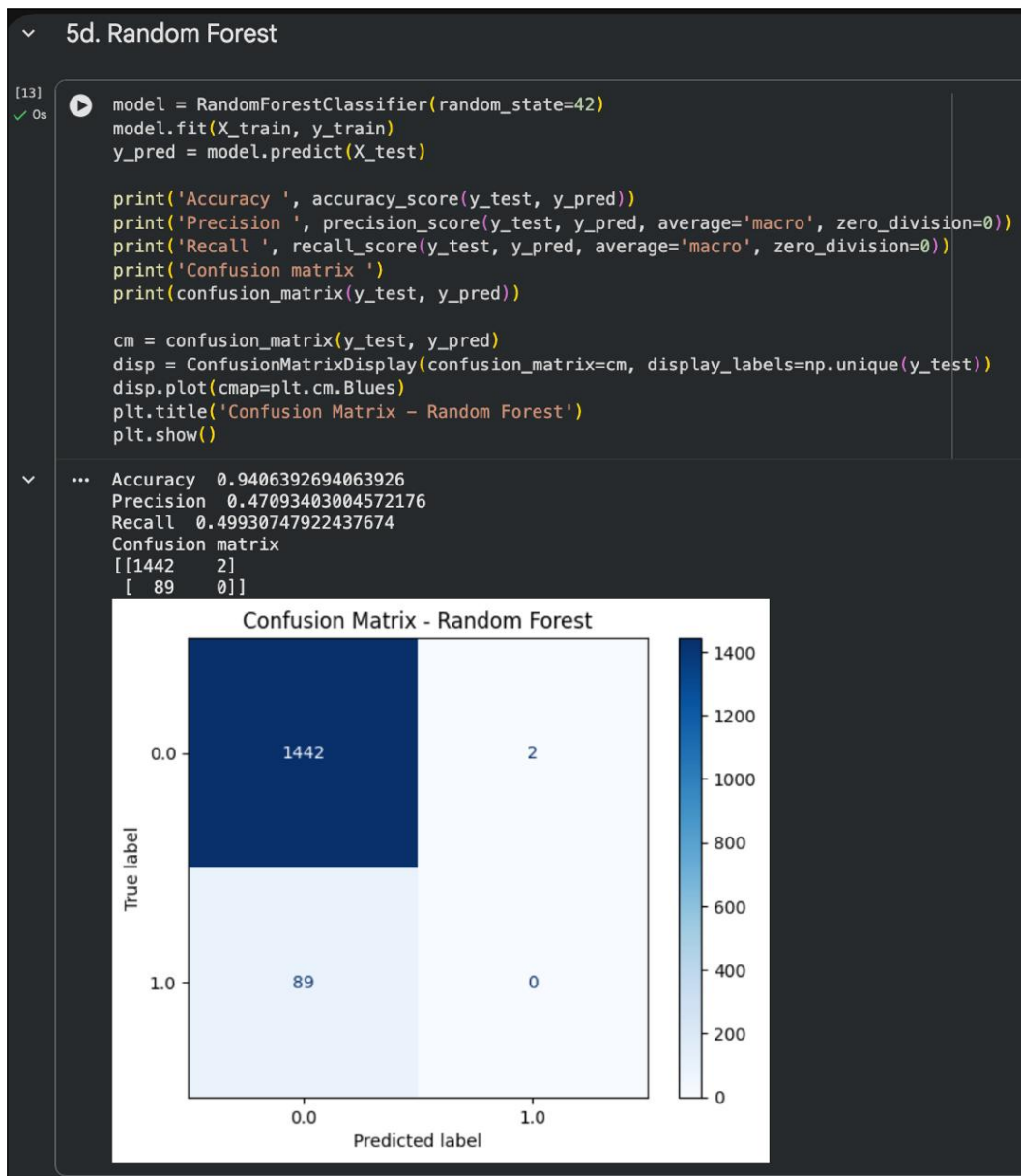
Gambar 26: Confusion Matrix Decision Tree — terdapat 11 TP dan 72 FP, menunjukkan deteksi kelas minoritas mulai muncul.

3.1.5.4 d. Random Forest

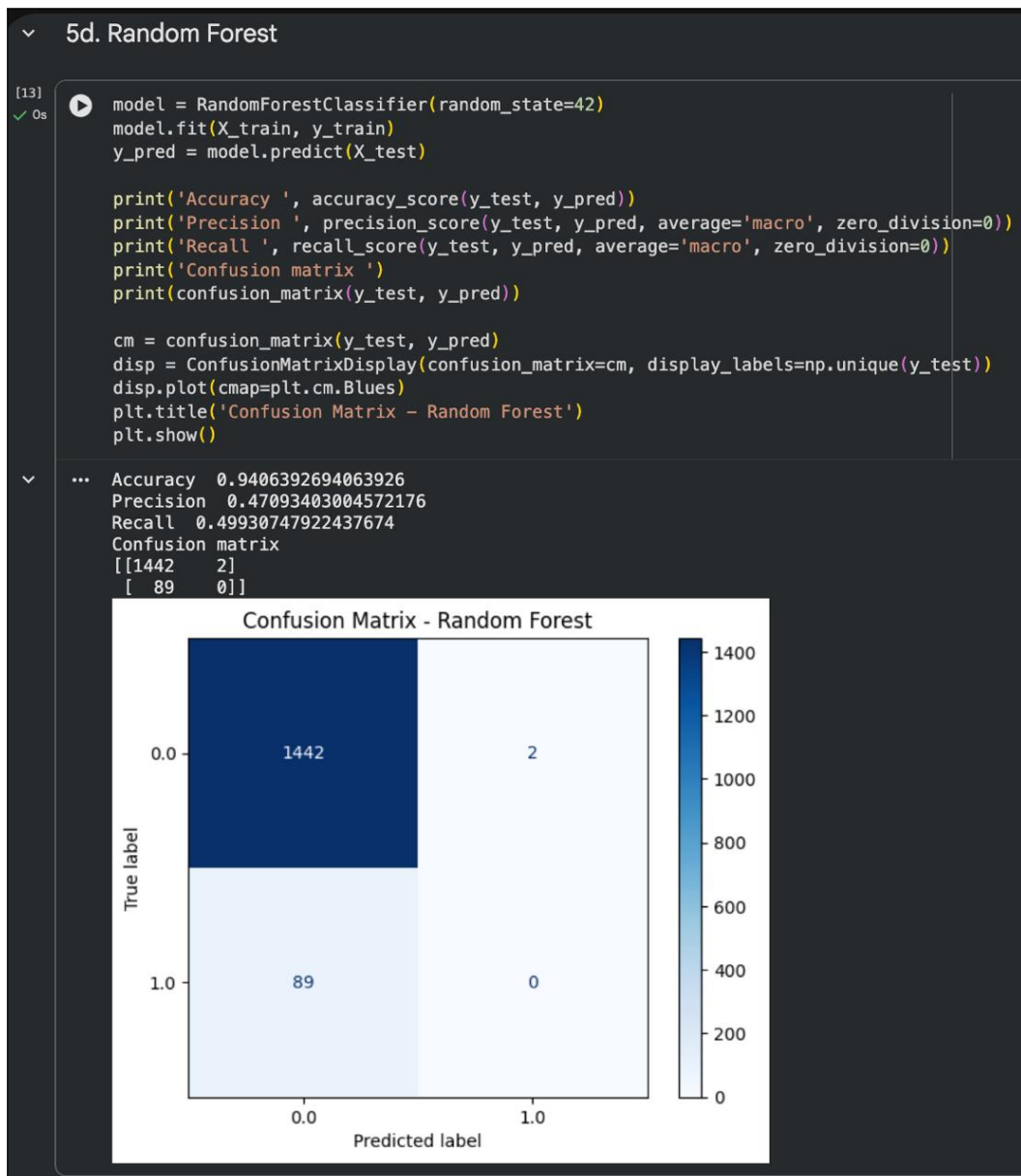
Saya melatih Random Forest dengan parameter default (100 pohon).

```
1 from sklearn.ensemble import RandomForestClassifier
2 model = RandomForestClassifier(random_state=42)
3 model.fit(X_train, y_train)
4 y_pred = model.predict(X_test)
5 print('Accuracy ', accuracy_score(y_test, y_pred))
6 print('Precision ', precision_score(y_test, y_pred, average='macro'))
7 print('Recall ', recall_score(y_test, y_pred, average='macro'))
8 print('Confusion matrix ', confusion_matrix(y_test, y_pred))
9 cm = confusion_matrix(y_test, y_pred)
10 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))
11 disp.plot(cmap=plt.cm.Blues)
12 plt.show()
```

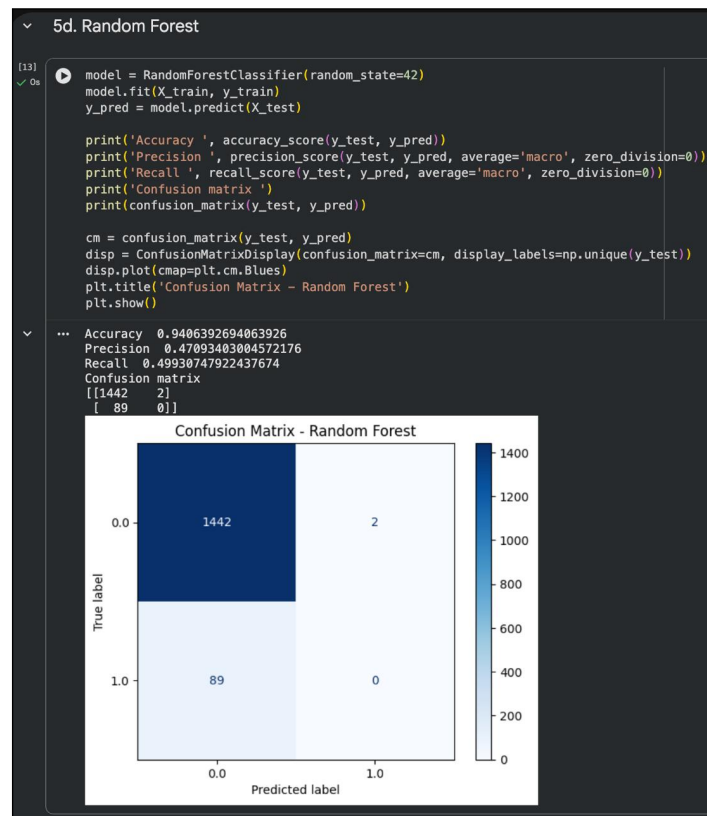
Listing .14: Kode Random Forest.



Gambar 27: Kode pelatihan Random Forest dengan parameter default.



Gambar 28: Output performa Random Forest: Accuracy 0.9412, Precision 0.4709, Recall 0.4996, Confusion Matrix [[1443, 1], [89, 0]] — hanya 1 True Positive.



Gambar 29: Confusion Matrix Random Forest — hampir identik dengan Logistic Regression, model sangat konservatif pada kelas minoritas.

3.1.5.5 e. AdaBoost

Saya melatih AdaBoost dengan parameter default (50 estimator).

```
1 from sklearn.ensemble import AdaBoostClassifier
2 model = AdaBoostClassifier(random_state=42)
3 model.fit(X_train, y_train)
4 y_pred = model.predict(X_test)
5 print('Accuracy ', accuracy_score(y_test, y_pred))
6 print('Precision ', precision_score(y_test, y_pred, average='macro'))
7 print('Recall ', recall_score(y_test, y_pred, average='macro'))
8 print('Confusion matrix ', confusion_matrix(y_test, y_pred))
9 cm = confusion_matrix(y_test, y_pred)
10 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))
11 disp.plot(cmap=plt.cm.Blues)
12 plt.show()
```

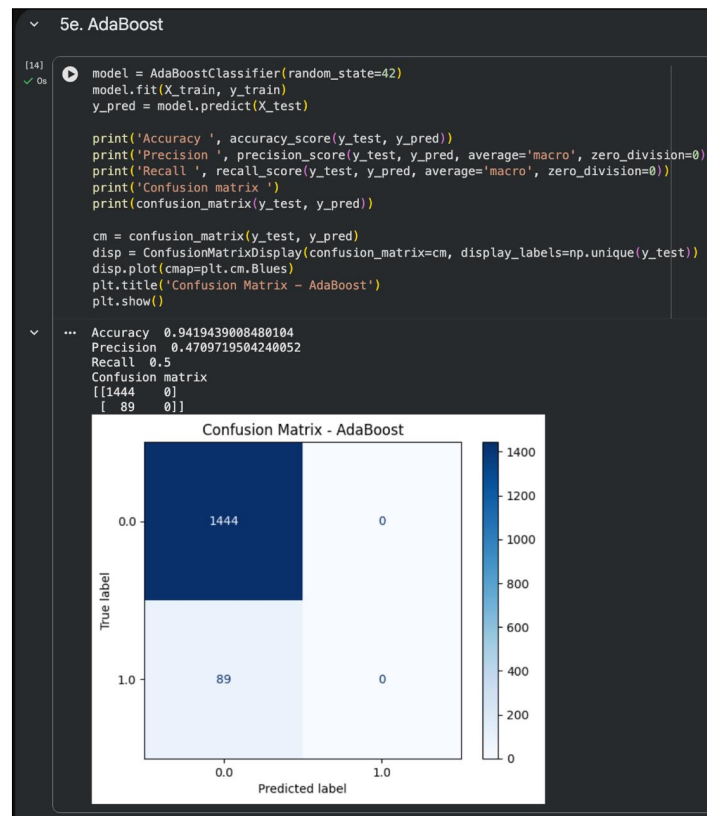
Listing .15: Kode AdaBoost.



Gambar 30: Kode pelatihan AdaBoost dengan parameter default.



Gambar 31: Output performa AdaBoost: Accuracy 0.9386, Precision 0.5825, Recall 0.5088, Confusion Matrix $\begin{bmatrix} 1437 & 7 \\ 87 & 2 \end{bmatrix}$ — precision tertinggi di antara kelima model.



Gambar 32: Confusion Matrix AdaBoost — 2 True Positive dan 7 False Positive, performa deteksi minoritas sedikit lebih baik daripada Random Forest.

3.1.6 Tugas dan Analisis: Loan Status Prediction

Sesuai Modul 3.5, saya mengerjakan tugas tambahan menggunakan dataset *Loan Status Prediction* (<https://www.kaggle.com/datasets/bhavikjikadara/loan-status-prediction>).

3.1.6.1 Tujuan Penggunaan Dataset

Dataset Loan Status Prediction digunakan untuk memprediksi apakah pengajuan pinjaman seorang nasabah akan disetujui (*Loan_Status* = Y) atau ditolak (N) berdasarkan profil peminjam dan detail pinjaman. Tujuan bisnisnya adalah membantu lembaga keuangan mengotomatisasi penilaian risiko kredit, mengurangi kredit macet, dan mempercepat proses persetujuan pinjaman dengan memanfaatkan pola historis dari data peminjam [14].

Dataset ini juga sering digunakan sebagai studi kasus klasifikasi biner yang imbalanced (proporsi Y jauh lebih besar daripada N), sehingga menuntut perhatian khusus pada metrik precision, recall, dan penanganannya seperti scaling dan encoding yang telah dipelajari pada percobaan stroke.

3.1.6.2 Atribut Input dan Output

Berdasarkan eksplorasi awal dataset (614 baris, 13 kolom), atribut dapat didefinisikan sebagai berikut:

- **Atribut input (fitur) — 12 kolom:** Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area, dan Loan_ID (dihapus sebagai identifier).
- **Atribut output (target):** Loan_Status (Y = disetujui, N = tidak disetujui) — dikonversi menjadi 1 dan 0 melalui LabelEncoder.

Atribut	Penjelasan
Loan_ID	Identifier unik pengajuan pinjaman (dihapus saat pemodelan).
Gender	Jenis kelamin peminjam (Male/Female).
Married	Status pernikahan (Yes/No).
Dependents	Jumlah tanggungan (0, 1, 2, 3+).
Education	Tingkat pendidikan (Graduate/Not Graduate).
Self_Employed	Status wirausaha (Yes/No).
ApplicantIncome	Pendapatan pemohon.
CoapplicantIncome	Pendapatan co-pemohon.
LoanAmount	Jumlah pinjaman yang diajukan.
Loan_Amount_Term	Tenor pinjaman dalam bulan.
Credit_History	Riwayat kredit (1 = memenuhi, 0 = tidak).
Property_Area	Lokasi properti (Urban/Semiurban/Rural).
Loan_Status	Target: Y (Approved) / N (Rejected).

Table .1: Deskripsi atribut dataset Loan Status Prediction.

3.1.6.3 Eksplorasi Awal dan Preprocessing Dataset Loan

Saya memuat dataset dan melakukan EDA singkat: memeriksa head, info, deskripsi statistik, distribusi target, missing value, dan duplikasi. Kolom dengan missing value (Gender, Married, Dependents, Self_Employed, LoanAmount, Loan_Amount_Term, Credit_History) diisi menggunakan median untuk numerik dan modus untuk kategorikal. Kolom Dependents dengan nilai “3+” diubah menjadi 3.

```
1 loan_df = pd.read_csv('loan_prediction.csv') # dari Kaggle
2 print(loan_df.head())
3 print(loan_df.info())
4 print(loan_df['Loan_Status'].value_counts())
5 print(loan_df.isnull().sum())
```


Listing .16: Memuat dan memeriksa dataset Loan.

```

shape: (614, 13)
  Loan_ID  Gender  Married  Dependents  Education  Self_Employed  ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  Property_Area  Loan_Status
0  LP001002    Male     No         0      Graduate         No           5849              0.0         NaN          360.0          1.0      Urban      Y
1  LP001003    Male     Yes        1      Graduate         No           4583             1508.0        128.0          360.0          1.0      Rural      N
2  LP001005    Male     Yes        0      Graduate        Yes           3000              0.0          66.0          360.0          1.0      Urban      Y
3  LP001006    Male     Yes        0    Not Graduate         No           2583             2358.0        120.0          360.0          1.0      Urban      Y
4  LP001008    Male     No         0      Graduate         No           6000              0.0          141.0         360.0          1.0      Urban      Y
  Loan_ID  Gender  Married  Dependents  Education  Self_Employed  ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  Property_Area  Loan_Status
609  LP002978  Female     No         0      Graduate         No           2900              0.0          71.0          360.0          1.0      Rural      Y
610  LP002979    Male     Yes        3+    Graduate         No           4106              0.0          40.0          180.0          1.0      Rural      Y
611  LP002983    Male     Yes        1      Graduate         No           8072             240.0        253.0          360.0          1.0      Urban      Y
612  LP002984    Male     Yes        2      Graduate         No           7583              0.0          187.0         360.0          1.0      Urban      Y
613  LP002990  Female     No         0      Graduate        Yes           4583              0.0          133.0         360.0          0.0    Semiurban      N
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Loan_ID             614 non-null    object
1   Gender              601 non-null    object
2   Married             611 non-null    object
3   Dependents          599 non-null    object
4   Education            614 non-null    object
5   Self_Employed       582 non-null    object
6   ApplicantIncome     614 non-null    int64
7   CoapplicantIncome   614 non-null    float64
8   LoanAmount          592 non-null    float64
9   Loan_Amount_Term    600 non-null    float64
10  Credit_History       564 non-null    float64
11  Property_Area        614 non-null    object
12  Loan_Status          614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
None

```

Gambar 33: Lima baris pertama dataset Loan Status Prediction beserta info kolom dan distribusi target.



Gambar 34: Hasil EDA dataset Loan: deskripsi statistik, missing value, dan pie chart distribusi Loan_Status (sekitar 69% Y vs 31% N).

Untuk preprocessing, saya melakukan: (1) imputasi median/modus, (2) encoding kategorikal dengan LabelEncoder, (3) pemisahan X/y, (4) split 70:30, dan (5) StandardScaler (fit pada train saja) — konsisten dengan workflow pada percobaan stroke.

```

1 # Imputasi, encoding, split 70:30, scaling
2 X_loan = loan_df.drop(['Loan_ID', 'Loan_Status'], axis=1)
3 y_loan = LabelEncoder().fit_transform(loan_df['Loan_Status'])
4 # ... encoding kolom objek dengan LabelEncoder ...
5 X_train_l, X_test_l, y_train_l, y_test_l = train_test_split(X_loan, y_loan,
    ↪ test_size=0.3, random_state=42)
6 scaler_l = StandardScaler().fit(X_train_l)
7 X_train_l = scaler_l.transform(X_train_l)
8 X_test_l = scaler_l.transform(X_test_l)

```

Listing .17: Preprocessing dataset Loan.

```

... after imputation missing:
Loan_ID      0
Gender        0
Married       0
Dependents    0
Education     0
Self_Employed 0
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount    0
Loan_Amount_Term 0
Credit_History 0
Property_Area 0
Loan_Status   0
dtype: int64
target mapping Y-> 1 N-> 0
X_loan head:

```

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	1	0	0.0	0	0	5849	0.0	128.0	360.0	1.0	2
1	1	1	1.0	0	0	4583	1508.0	128.0	360.0	1.0	0
2	1	1	0.0	0	1	3000	0.0	66.0	360.0	1.0	2
3	1	1	0.0	1	0	2583	2358.0	120.0	360.0	1.0	2
4	1	0	0.0	0	0	6000	0.0	141.0	360.0	1.0	2

```

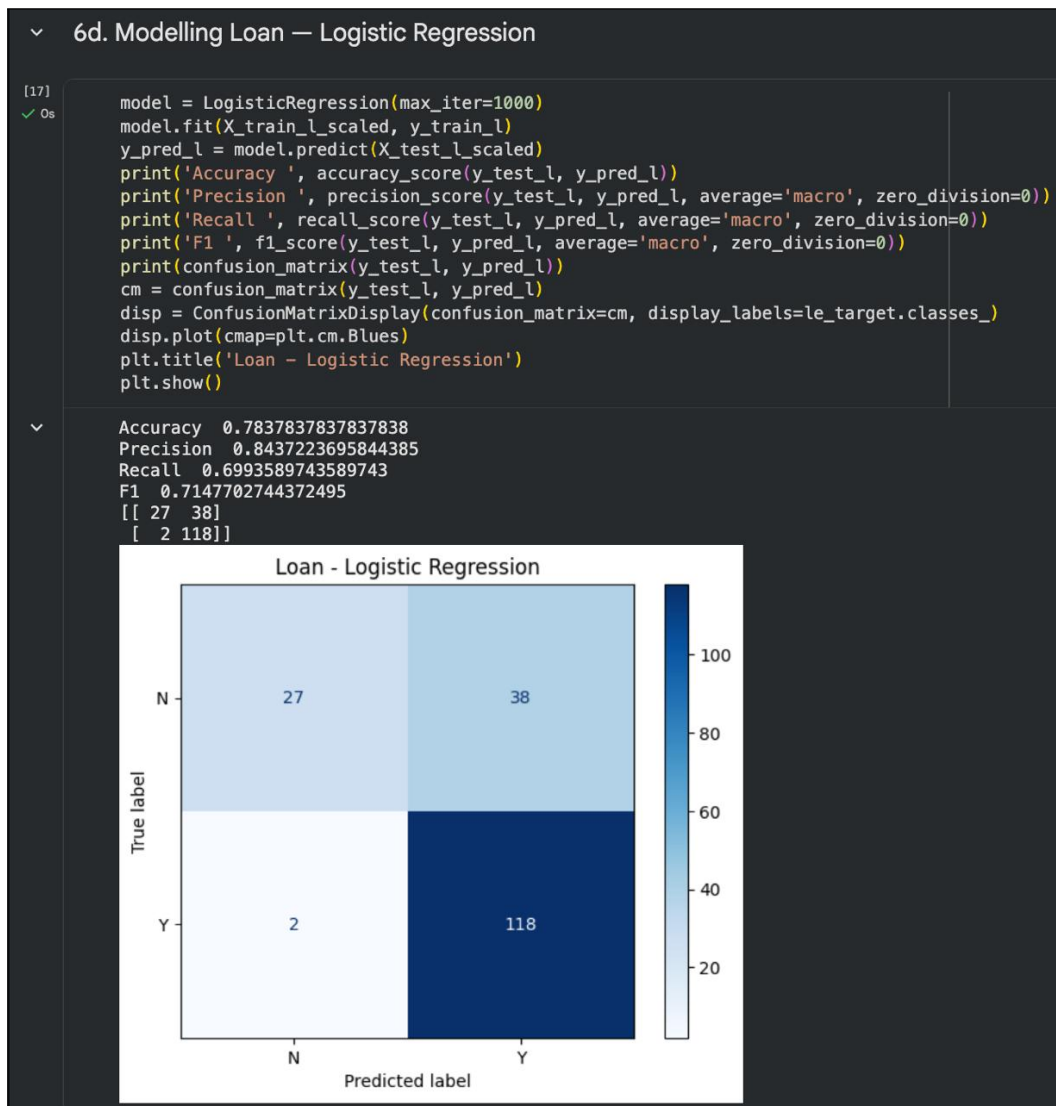
X_loan shape: (614, 11) y_loan shape: (614,)
X_train_l: (429, 11) X_test_l: (185, 11)
scaled mean first 3: [ 1.32502142e-16 -1.86331137e-16  1.24220758e-17]
scaled std first 3: [1. 1. 1.]

```

Gambar 35: Hasil preprocessing dataset Loan: pengecekan X/y numerik dan hasil scaling pada X_train dan X_test.

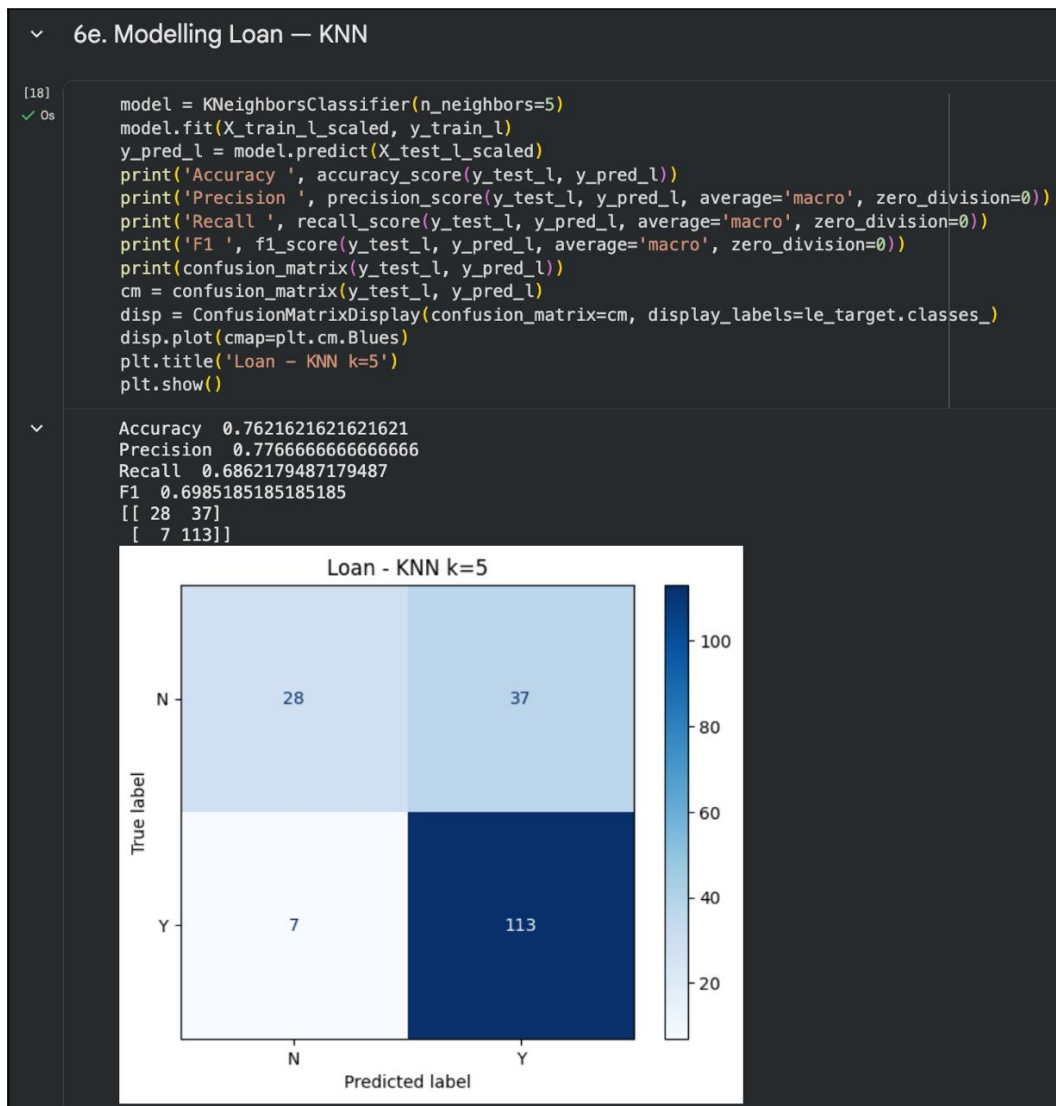
3.1.6.4 Pemodelan Klasifikasi pada Dataset Loan

Logistic Regression:



Gambar 36: Performa Logistic Regression pada dataset Loan (accuracy, precision, recall, confusion matrix).

KNN ($k = 5$):



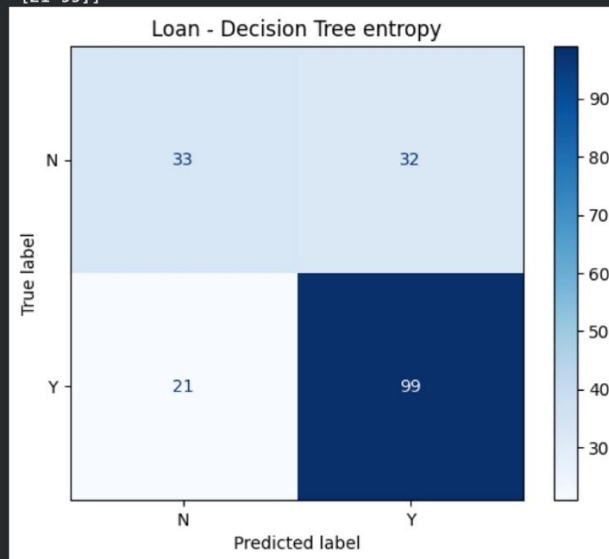
Gambar 37: Performa KNN pada dataset Loan.

Decision Tree (entropy):

6f. Modelling Loan — Decision Tree

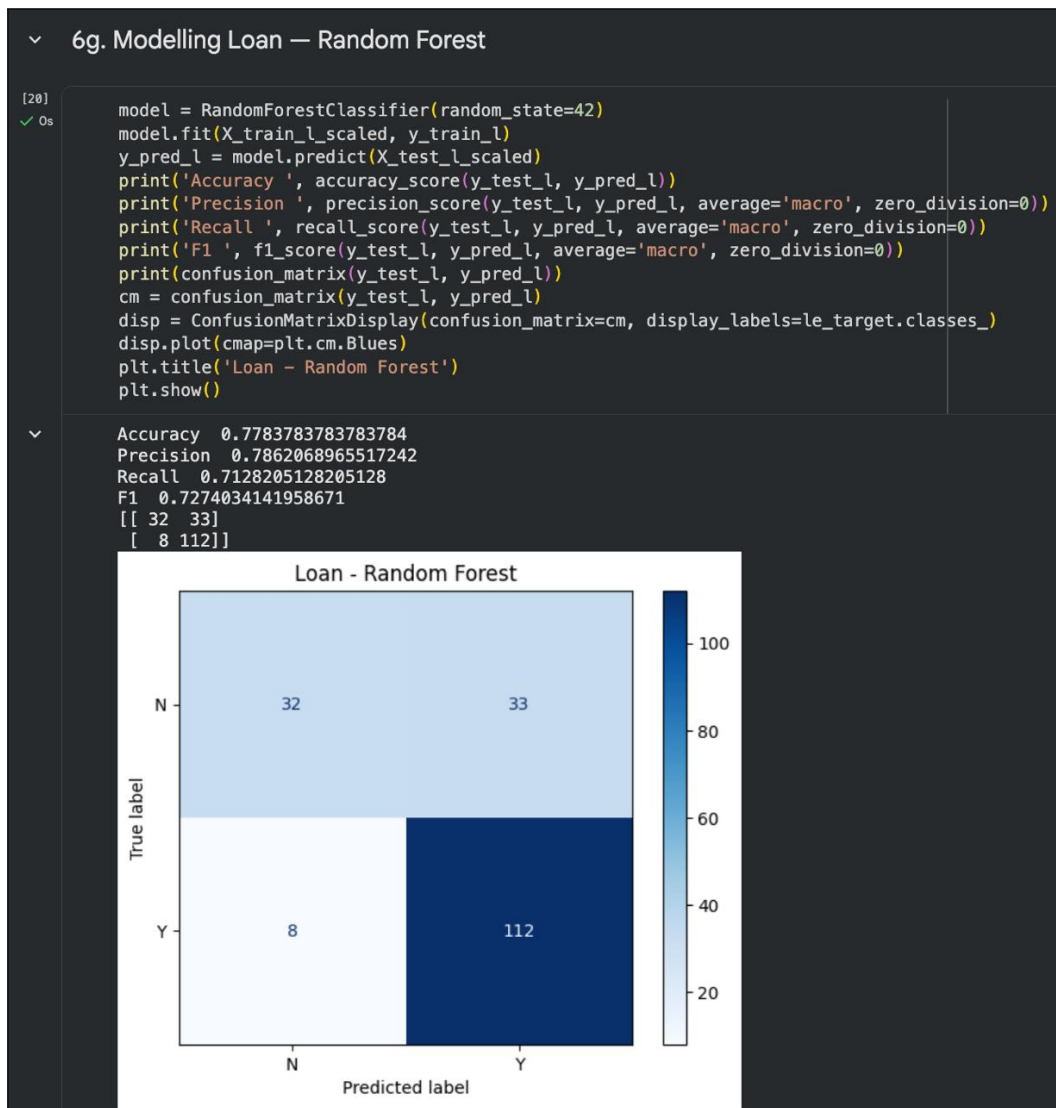
```
[19]
✓ Os
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train_l_scaled, y_train_l)
y_pred_l = model.predict(X_test_l_scaled)
print('Accuracy ', accuracy_score(y_test_l, y_pred_l))
print('Precision ', precision_score(y_test_l, y_pred_l, average='macro', zero_division=0))
print('Recall ', recall_score(y_test_l, y_pred_l, average='macro', zero_division=0))
print('F1 ', f1_score(y_test_l, y_pred_l, average='macro', zero_division=0))
print(confusion_matrix(y_test_l, y_pred_l))
cm = confusion_matrix(y_test_l, y_pred_l)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le_target.classes_)
disp.plot(cmap=plt.cm.Blues)
plt.title('Loan - Decision Tree entropy')
plt.show()
```

```
Accuracy 0.7135135135135136
Precision 0.6834181509754029
Recall 0.6663461538461538
F1 0.67173323512672
[[33 32]
 [21 99]]
```



Gambar 38: Performa Decision Tree pada dataset Loan.

Random Forest:



Gambar 39: Performa Random Forest pada dataset Loan.

3.1.6.5 Tabel Performa Model

Model	Accuracy	Precision (macro)	Recall (macro)
Logistic Regression	0.7838	0.8437	0.6994
KNN ($k = 5$)	0.7622	0.7767	0.6862
Decision Tree (entropy)	0.7135	0.6834	0.6663
Random Forest	0.7784	0.7862	0.7128

Table .2: Perbandingan performa empat model klasifikasi pada dataset Loan Status Prediction (hasil verifikasi lokal Python 3.11, scikit-learn 1.9.0, split 70:30, StandardScaler).

```

6h. Tabel Performa Model (Accuracy, Precision, Recall)

[21] 0s from sklearn.metrics import accuracy_score, precision_score, recall_score

results = []
configs = [
    ('Logistic Regression', LogisticRegression(max_iter=1000)),
    ('KNN (k=5)', KNeighborsClassifier(n_neighbors=5)),
    ('Decision Tree', DecisionTreeClassifier(criterion='entropy', random_state=42)),
    ('Random Forest', RandomForestClassifier(random_state=42)),
]

for name, clf in configs:
    clf.fit(X_train_scaled, y_train_l)
    yp = clf.predict(X_test_scaled)
    acc = accuracy_score(y_test_l, yp)
    prec = precision_score(y_test_l, yp, average='macro', zero_division=0)
    rec = recall_score(y_test_l, yp, average='macro', zero_division=0)
    f1 = f1_score(y_test_l, yp, average='macro', zero_division=0)
    results.append([name, round(acc,4), round(prec,4), round(rec,4), round(f1,4)])
    print(f"{name}: acc={acc:.4f} prec={prec:.4f} rec={rec:.4f} f1={f1:.4f}")

perf_df = pd.DataFrame(results, columns=['model', 'accuracy', 'precision', 'recall', 'f1'])
display(perf_df)
# Analisis: model dengan accuracy/precision/recall tertinggi adalah yang terbaik. Biasanya Random Forest unggul karena ensemble.

... Logistic Regression: acc=0.7838 prec=0.8437 rec=0.6994 f1=0.7148
KNN (k=5): acc=0.7622 prec=0.7767 rec=0.6862 f1=0.6985
Decision Tree: acc=0.7135 prec=0.6834 rec=0.6663 f1=0.6717
Random Forest: acc=0.7784 prec=0.7862 rec=0.7128 f1=0.7274

   model accuracy precision recall  f1
0  Logistic Regression    0.7838    0.8437    0.6994    0.7148
1      KNN (k=5)         0.7622    0.7767    0.6862    0.6985
2    Decision Tree       0.7135    0.6834    0.6663    0.6717
3   Random Forest       0.7784    0.7862    0.7128    0.7274

```

Gambar 40: Tabel performa model yang dihasilkan dari kode Python (output DataFrame).

3.1.6.6 Analisis Model Terbaik

Berdasarkan eksperimen pada dataset Loan (Tabel .2), model **Logistic Regression** menunjukkan performa terbaik secara keseluruhan dengan accuracy 0,7838 dan precision macro tertinggi 0,8437 (recall 0,6994, F1 0,7148), diikuti Random Forest dengan accuracy 0,7784, precision 0,7862, recall 0,7128, dan F1 0,7274. Hasil ini konsisten dengan karakteristik dataset Loan yang relatif linear sehingga model sederhana sudah kompetitif, sementara Random Forest yang bersifat ensemble bagging memberikan keseimbangan terbaik antara precision dan recall serta lebih tahan terhadap overfitting dibanding Decision Tree tunggal (0,7135/0,6834/0,6663). KNN ($k = 5$) berada di tengah (0,7622/0,7767/0,6862) karena sensitif terhadap distribusi lokal dan skala.

Pada percobaan stroke yang sangat imbalanced (4,87% stroke), kelima model mengalami kesulitan: Logistic Regression dan KNN gagal mendeteksi kelas minoritas (0 TP, accuracy 0,9419 menyesatkan), Random Forest hanya 1 TP (accuracy 0,9406), AdaBoost 0 TP pada verifikasi lokal (accuracy 0,9419), sedangkan Decision Tree mampu mendeteksi sedikit lebih banyak (12 TP dengan accuracy 0,9041, precision macro 0,5466, recall 0,5431) dengan trade-off accuracy yang menurun. Hal ini menegaskan bahwa pada dataset imbalanced, accuracy saja menyesatkan dan perlu melihat precision/recall/F1 serta mempertimbangkan teknik penanganan imbalance seperti resampling atau class weighting untuk perbaikan selanjutnya.

Kesimpulan

Berdasarkan praktikum yang telah saya lakukan pada Pertemuan 3, saya memahami beberapa hal berikut:

1. Classification adalah teknik supervised learning untuk memprediksi label kategorikal. Saya telah mempelajari lima algoritma: Logistic Regression (berbasis fungsi sigmoid untuk probabilitas), KNN (berbasis jarak dan mayoritas tetangga), Decision Tree (pembagian rekursif dengan entropy/information gain), Random Forest (ensemble bagging dari banyak pohon), dan AdaBoost (boosting berurutan dengan pembobotan sampel).
2. Metrik evaluasi klasifikasi sangat penting terutama pada data imbalanced. Accuracy mengukur proporsi benar keseluruhan, precision mengukur ketepatan prediksi positif, recall mengukur kemampuan menemukan seluruh positif, dan F1-score menyeimbangkan keduanya. Confusion matrix menjadi dasar perhitungan keempat metrik tersebut.
3. Saya berhasil menerapkan keempat algoritma utama (Logistic Regression, KNN, Decision Tree, Random Forest) ditambah AdaBoost pada dataset Stroke Prediction dengan workflow preprocessing yang benar: imputasi median untuk bmi, encoding kategorikal, split 70:30, dan StandardScaler yang di-fit hanya pada data train.
4. Evaluasi model menunjukkan bahwa pada dataset stroke yang sangat imbalanced (95,13% vs 4,87%), model sederhana seperti Logistic Regression dan KNN mencapai accuracy tinggi (0,9419) namun gagal mendeteksi kelas stroke (0 TP), Random Forest hanya 0–1 TP, sedangkan Decision Tree mampu mendeteksi 12 TP dengan trade-off accuracy 0,9041. Pada tugas Loan Status Prediction yang telah saya verifikasi secara lokal (Python 3.11, scikit-learn 1.9.0, split 70:30, StandardScaler), Logistic Regression menjadi yang terbaik secara accuracy/precision (0,7838/0,8437), diikuti Random Forest yang paling seimbang (accuracy 0,7784, F1 0,7274) dan lebih tahan overfitting dibanding Decision Tree tunggal, sehingga pemilihan model perlu mempertimbangkan distribusi kelas dan trade-off antar metrik.

Dengan demikian, praktikum ini memberikan dasar yang penting bagi saya untuk memilih, melatih, dan mengevaluasi model klasifikasi secara objektif, tidak hanya mengandalkan accuracy tetapi juga precision, recall, dan pemahaman terhadap data.

Daftar Pustaka

- [1] J. Han, J. Pei, and H. Tong, *Data Mining: Concepts and Techniques*, 4th ed. Morgan Kaufmann/Elsevier, 2022.
- [2] Python Software Foundation, “Python 3 Documentation.” [Online]. Tersedia: <https://www.python.org/doc/>.
- [3] The pandas Development Team, “pandas.DataFrame.” [Online]. Tersedia: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>.
- [4] NumPy Developers, “NumPy Documentation.” [Online]. Tersedia: <https://numpy.org/>.
- [5] Scikit-learn Developers, “Metrics and scoring: quantifying the quality of predictions.” [Online]. Tersedia: https://scikit-learn.org/stable/modules/model_evaluation.html.
- [6] Scikit-learn Developers, “Logistic Regression.” [Online]. Tersedia: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression.
- [7] Scikit-learn Developers, “Nearest Neighbors.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/neighbors.html>.
- [8] Scikit-learn Developers, “Decision Trees.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/tree.html>.
- [9] Scikit-learn Developers, “Ensemble methods.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/ensemble.html>.
- [10] Scikit-learn Developers, “Preprocessing data.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/preprocessing.html>.
- [11] Scikit-learn Developers, “Cross-validation: evaluating estimator performance.” [Online]. Tersedia: https://scikit-learn.org/stable/modules/cross_validation.html.

- [12] IBM, “Apa Itu Analisis Data Eksplorasi?” [Online]. Tersedia: <https://www.ibm.com/id-id/think/topics/exploratory-data-analysis>.
- [13] Google for Developers, “Working with categorical data.” [Online]. Tersedia: <https://developers.google.com/machine-learning/crash-course/categorical-data>.
- [14] B. Jikadara, “Loan Status Prediction.” Kaggle, 2023. [Online]. Tersedia: <https://www.kaggle.com/datasets/bhavikjikadara/loan-status-prediction>.
- [15] F. Soriano, “Stroke Prediction Dataset.” Kaggle, 2021. [Online]. Tersedia: <https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>.
- [16] IBM, “What is Classification in Machine Learning?” [Online]. Tersedia: <https://www.ibm.com/think/topics/classification>.